IACR Transactions on Cryptographic Hardware and Embedded Systems
ISSN XXXX-XXXX, Vol. 0, No. 0, pp. 1–41. DOI:XXXXXXXX

Exploiting noisy single-bit leakage in ML-DSA

Kaveh Bashiri, Jan Geuenich and Johannes Mittmann

Bundesamt für Sicherheit in der Informationstechnik (BSI), Bonn, Germany
firstname.lastname@bsi.bund.de

Abstract. ML-DSA implementations face a serious risk from partial leakage of the mask vector $\boldsymbol{y}$. Recent research has shown that this threat is practical. Even highly noisy, single-bit leakage accumulated over many signatures can suffice to recover the secret key. We carefully analyze the number of signatures with bit leakage required for successful key recovery using a stochastic model, rather than relying on a concrete attack method. On the practical side, we develop new attack methods capable of recovering the key using almost the minimal number of signatures required in theory. Our attacks work for bit-error probabilities as high as 0.49 and for leakage at every bit position of index at least four or five (depending on the ML-DSA parameter set). In the most favorable scenario, leakage at bit index four keeps our attack practical for leaked bits with an error probability of 0.499, and in the absence of noise reduces the signature requirement to below 1000.

Keywords: ML-DSA · Dilithium · key recovery · integer linear programming · nonlinear programming

1 Introduction

A major challenge in the transition to post-quantum cryptography is understanding and addressing side-channel vulnerabilities in the newly standardized algorithms. Research on the signature scheme ML-DSA has highlighted the importance of preventing leakage from the mask vector, even of single noisy bits; see, e.g., [Liu+21; Wan+21; Dam+25a]. In this work, we contribute to this line of research by further investigating a leakage attack, introduced in [Liu+21] and subsequently improved in [Dam+25a; Dam+26]. In parallel to our work, [Sch+26] enhanced the attack via a nonlinear approach. Despite methodological similarities, their results indicate that our methods require fewer signatures and cover more bit positions, though refinements on their side may narrow the gap.

The investigation we carry out in this paper is entirely theoretical; specifically, we do not address the practical mechanisms by which leakage from the mask vector might be extracted in real-world implementations. In this direction, [Qia+23], motivated by [Liu+21], conducted experiments on both unprotected and masked ML-DSA implementations on a microprocessor, targeting the mask vector through power analysis. Further experiments to obtain leakage from the mask vector via power analysis were reported in [Qia+24]. More recently, for masked implementations of ML-DSA running on a microcontroller, [HNP25] presents a template attack targeting individual bits of the mask vector, while [Dam+25b] demonstrates exploitable leakage of sign bits. Although we focus on single-bit leakage in this paper, it is worth noting that single bits are often more difficult to target in practice than specific types of combined leakage of multiple bits (e.g., Hamming-weight leakage).

Mask-vector leakage in ML-DSA

ML-DSA signatures consist, aside from auxiliary data not relevant here, of tuples $(c, \boldsymbol{z})$, where c is a polynomial and $\boldsymbol{z} = c\,\boldsymbol{s}_1 + \boldsymbol{y}$ a vector with ℓ components, each of which is a

Licensed under Creative Commons License CC-BY 4.0.

polynomial with 256 integer coefficients. It is crucial that the mask vector $\boldsymbol{y}$ is not revealed with the signature, since otherwise the vector $\boldsymbol{s}_1$, which is the core part of the secret key, could be recovered. Furthermore, leakage of coefficients of components $\boldsymbol{y}_i$ of the mask vectors across different signatures must be prevented, as this would yield a system of linear equations that may be solvable for the components $\boldsymbol{s}_{1,i}$ of the secret key. The works of [Liu+21; Dam+25a] go further, demonstrating that even repeated leakage of a single noisy bit of one of the coefficients of $\boldsymbol{y}_i$ across different signatures can be used to recover $\boldsymbol{s}_{1,i}$.

Characteristics of the leakage model

The model considered in these works, and further investigated in this paper, assumes access to a sequence of ML-DSA signatures, generated with a single key, together with, for each signature, a leaked bit from the bit representation of one coefficient of the mask vector. We assume that the index λ of the bit within the coefficient's binary representation is fixed. This assumption simplifies the theoretical analysis, although the attacks themselves can be readily adapted to handle varying bit positions. The model allows for noise, i.e., the leaked bit may be incorrect with some error probability p_{err}.

For the rest of the paper, we consider a simplified notion of a signature as a tuple (c, z), where z denotes the integer value of the coefficient of $\boldsymbol{z}_i$ from which the leakage originates (and where c is transformed by a "signed rotation"). This will be made precise and justified later. It was observed in [Dam+25a] that bit leakage at index λ from the corresponding coefficient of the mask vector can reveal information about the secret key only for certain values of z. This observation naturally leads to the notion of informative signatures.

The bit indices split into two regimes, which we refer to as the low-λ and high-λ regimes. For ML-DSA-44 and ML-DSA-87 the low-λ regime consists of indices below 8, and for ML-DSA-65 of indices below 9. For informative signatures, the leakage models are uniform in the high-λ regime, meaning that, on average, the attacker faces a problem of equal hardness for every index in this regime. Consequently, for the analysis it suffices to restrict attention to a single representative bit index from the high-λ regime, while in the low-λ regime every index requires separate examination.

For comparison, we note that, assuming bit λ of each coefficient of $\boldsymbol{y}$ leaks, every ML-DSA signature yields, for each of the ℓ components of $\boldsymbol{s}_1$, an expected $256\,p_{\mathsf{inf}}$ informative signatures in the sense of this paper; see Figure 3 for the values of the probability p_{inf} that a signature is informative (in particular, observe that $p_{\mathsf{inf}} = 1$ in the low-λ regime). While our theoretical analysis is based on the assumption that every signature leaks only a single bit, our attacks can also be applied to several bit leaks per signature in practice with straightforward modifications. For example, for ML-DSA-44 with $\lambda = 4$, assuming one bit of exact leakage per coefficient of $\boldsymbol{y}$, four full ML-DSA signatures suffice to recover the vector $\boldsymbol{s}_1$, the part of the ML-DSA key required for forging signatures.

Contributions

We introduce a stochastic model that formalizes the leakage exhibited by the signatures and characterize the set of keys that maximize the attacker's success probability, enabling the development of tailored attacks. We then analyze how much information they reveal on average about the key. Finally, we establish lower and upper bounds on the expected number of signatures with bit leakage that are required to ensure that only a single key could have generated them. These bounds are proven rigorously in the high-λ regime and provide good heuristics in the low-λ regime. The tools used in our theoretical results rely on classical inequalities and approaches from probability theory. As such, our theoretical findings contribute valuable insights, even from a purely probabilistic standpoint.

With this foundation, we propose new practical methods for key recovery. We begin by formulating the problem as an integer linear program (ILP). In the case of exact leakage,

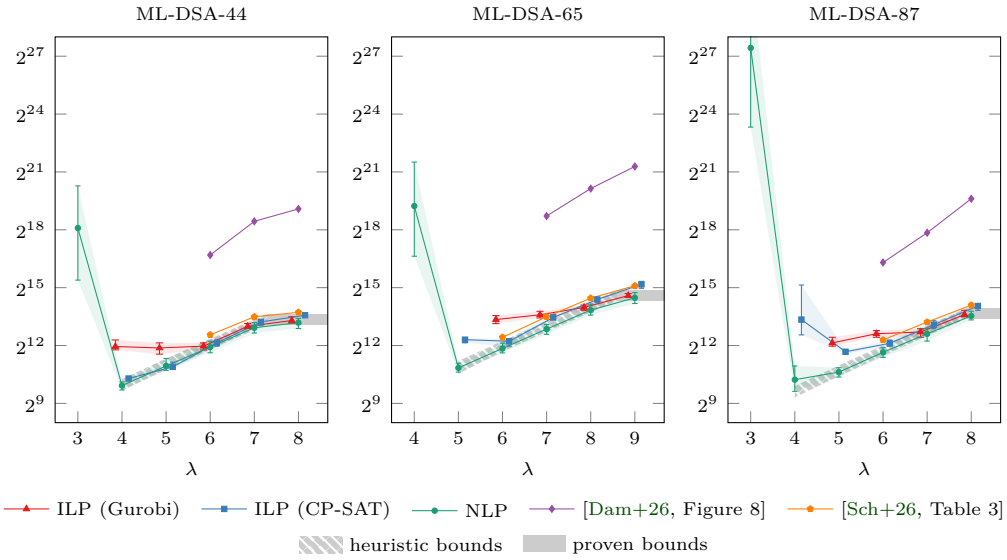


Figure 1: Summary of results for exact leakage; number of informative signatures needed.

we solve it using the optimizers Gurobi and CP-SAT: Gurobi turns out to be more efficient for high bit indices, whereas CP-SAT performs better for low bit indices. We then develop a nonlinear-programming (NLP) attack that combines a continuous gradient-descent stage with a subsequent combinatorial search. In the experiments, NLP outperforms ILP both in runtime and the number of signatures required. This yields an improvement of a factor of 25–100 over the attacks reported in [Dam+26]. Figure 1 displays our theoretical bounds and experimental results in the case of exact leakage, contrasted with results previously reported in the literature. As the experiments may differ in methodology, hardware, and implementation, the comparison should be interpreted cautiously. The figure reveals that our attack methods are nearly optimal in the sense that the number of signatures they require is close to the theoretical lower bound. It is worth noting that our attacks succeed for bit positions down to $\lambda = 4$ for ML-DSA-44 and ML-DSA-87, and down to $\lambda = 5$ for ML-DSA-65 with a reasonable number of signatures. The attack continues to work for $\lambda = 3$ and $\lambda = 4$, respectively, but then requires a vast number of signatures, and is therefore only practical when the bit-error probability is very low.

Our NLP attack is still highly effective even under substantial noise: bit-error probabilities as high as 0.49 are manageable for all bit indices λ that allow key recovery under exact leakage. Figure 2 presents our theoretical upper-bound estimate and experimental results for ML-DSA-44 in the high-λ regime, alongside previous results reported in the literature. It shows that the NLP attack remains near-optimal for noisy leakage, with the number of signatures required close to our heuristic estimate (for the highest error probabilities we did not minimize the number of signatures but only verified for each data point that key recovery was possible in all experiments with the proven upper bound).

In the most favorable situation for the attacker (ML-DSA-44 with $\lambda = 4$), our best attack requires on average fewer than 1000 signatures with noiseless bit leakage, and our implementation tolerates bit-error probabilities as high as 0.499 in the noisy setting.

Related work

There is a significant body of prior work on different leakage models for the mask vector $\boldsymbol{y}$. We briefly mention the most closely related works below.

Single-bit leakage. [Liu+21] were the first to exploit leakage of single noisy bits of the mask vector. They reduced the problem to ILWE and solved it using linear least-squares

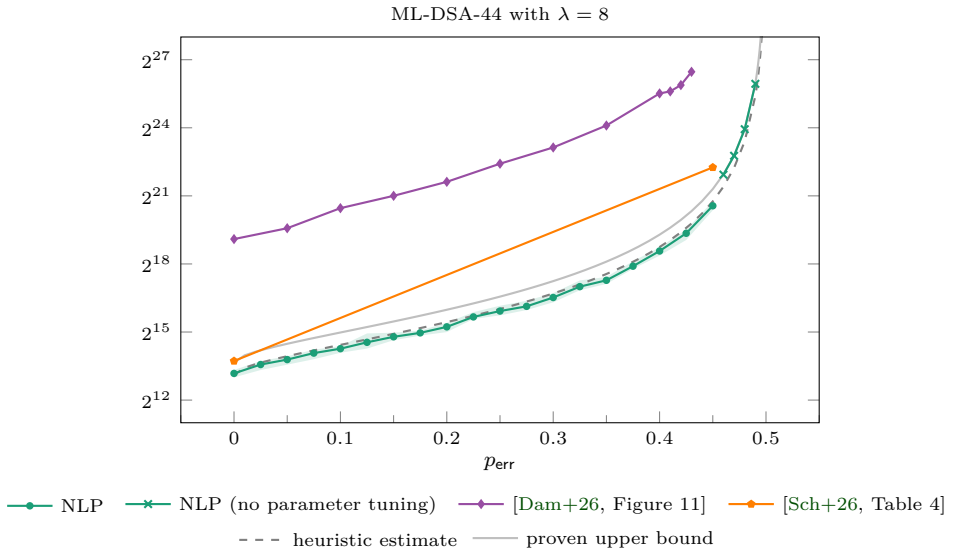


Figure 2: Summary of results for noisy leakage; number of informative signatures needed.

regression. The paper [Wan+21] improved this approach and, in particular, extended it to cover the lower bit indices $\lambda = 4$ and $\lambda = 5$ for ML-DSA-87 in the case of exact leakage. Further improvements were obtained by [Dam+25a; Dam+26], which introduced the important technique of filtering for informative signatures. Parallel to our work, [Sch+26] used a nonlinear method, which just as our work significantly reduces the number of required signatures. For a quantitative comparison of our results with the most recent works [Dam+26; Sch+26] see Figure 1 and Figure 2.

Sign-bit leakage. [BKM26] considered (among other things) leakage of the sign bit of mask-vector coefficients. This is a special case of single-bit leakage if the two's complement representation is used. Indeed, the sign bit corresponds to bits of index $\lambda \geq 17$ or $\lambda \geq 19$ (depending on the ML-DSA parameter set). They used linear programming and logistic regression for their attacks. Based on their reported results, our attacks require fewer signatures by a moderate factor.

Zero-coefficient leakage. The work [Uli+22] considers leakage where for some mask-vector coefficients the information whether they are zero leaks with high confidence. It employs a combination of linear least-squares regression and integer linear programming for key recovery. The paper [Dam+25b] improves on exploiting this kind of leakage by using Huber and Cauchy regression, which tolerate a substantially higher number of coefficients being erroneously classified as zero.

General leakage models. [Bro+24] and [HNP25] introduce frameworks to deal with more general leakage of mask-vector coefficients using belief propagation (BP). These papers mainly report on experiments for Hamming-weight leakage. The work in [HNP25] builds upon [Her+25], also targets masked implementations, and additionally investigates leakage of (multiple) individual bits. As proposed by the anonymous reviewers and stated as future research direction in [Sch+26], it is possible to use BP also to exploit single-bit leakage for key recovery. Motivated by this, we carried out some quick experiments using the BP implementation of [Her+25]. For instances requiring a moderate number of leaky signatures, the results were very similar to those achieved with our attacks; see Remark 33.

Outline

Section 2 sets out the notation, a simplified version of ML-DSA, and the bit representation assumed throughout this paper. The problem investigated in this work will be formally

defined and justified in Section 3. Section 4 summarizes our key theoretical results, including a characterization of optimal solutions to the problem under consideration and data complexity estimates for this problem expressed in terms of the number of required signatures, both in the presence and absence of noise. Formal definitions and proofs are deferred to Appendix A, including the definition and analysis of a stochastic model on which our theoretical results are based; the appendix closes with some open problems. Section 5 and Section 6 contain our attacks based on integer linear programming and nonlinear programming, respectively. Section 7 concludes this paper.

2 Preliminaries

2.1 Notation

For $n \geq 0$, we define $[n] \coloneqq \{1, \ldots, n\}$. The power set of a set $\mathcal{A}$ is written $2^{\mathcal{A}}$. For $\tau \geq 0$, the set of all τ-subsets of a finite set $\mathcal{A}$ is denoted by $\binom{\mathcal{A}}{\tau}$.

Let $m \geq 1$ be a modulus and let $a \in \mathbb{Z}$. We write $a \bmod m$ for the unique element of $[0, m) \cap \mathbb{Z}$ that is congruent to a modulo m. Moreover, we write $a \bmod^{\pm} m$ and $a \bmod^{\mp} m$ for the unique elements of $(-m/2, m/2] \cap \mathbb{Z}$ and $[-m/2, m/2) \cap \mathbb{Z}$, respectively, that are congruent to a modulo m. We extend these definitions to any $a \in \mathbb{R}$ by setting $a \bmod m \coloneqq a - \lfloor a/m \rfloor \cdot m$, $a \bmod^{\pm} m \coloneqq a - \lceil (a - m/2)/m \rceil \cdot m$, and $a \bmod^{\mp} m \coloneqq a - \lfloor (a + m/2)/m \rfloor \cdot m$.

Remark 1. The operator $\bmod^{\pm}$ is used in the specification of ML-DSA (see Section 2.2), while $\bmod^{\mp}$ will be used in conjunction with the bit representation we assume in this paper (see Section 2.3).

2.2 ML-DSA

The lattice-based digital signature scheme ML-DSA is standardized in FIPS 204 [FIPS204]. It is based on the latest specification [Dilithium] of CRYSTALS-Dilithium [Duc+18]. In this paper, we focus on an abbreviated and simplified version of ML-DSA, omitting technical details irrelevant to our attacks. We now briefly recall the setting and this simplified version of the signature scheme; see also [Dilithium, Section 1.1]. In the following description, we freely use the notation in [FIPS204]. We also refer to [Lyu09; Lyu24] for further background on ML-DSA.

For all parameter sets (see Table 1), ML-DSA works over the ring $R \coloneqq \mathbb{Z}[X]/(X^{256} + 1)$ and its modular reduction $R_q \coloneqq \mathbb{Z}_q[X]/(X^{256} + 1)$, where $q = 2^{23} - 2^{13} + 1 = 8\,380\,417$.

Table 1: ML-DSA parameter sets (excerpt) [FIPS204].

Parameters	**Values**		
	ML-DSA-44	ML-DSA-65	ML-DSA-87
(k, ℓ)	$(4, 4)$	$(6, 5)$	$(8, 7)$
η	2	4	2
$\gamma = \gamma_1$	2^{17}	2^{19}	2^{19}
γ_2	$(q-1)/88$	$(q-1)/32$	$(q-1)/32$
τ	39	49	60
$\beta = \tau \cdot \eta$	78	196	120

Key generation (see Algorithm 1) samples two secret vectors $\boldsymbol{s}_1$ and $\boldsymbol{s}_2$, whose entries are polynomials with small coefficients drawn from $S_\eta \coloneqq \{w \in R \mid \|w\|_\infty \leq \eta\}$, and a public matrix $\boldsymbol{A}$ over R_q. Signature generation (see Algorithm 2) and verification (see Algorithm 3) are constructed via a Fiat–Shamir transform applied to a Σ-protocol that

proves knowledge of secret vectors equivalent to $\boldsymbol{s}_1$ and $\boldsymbol{s}_2$. During signature generation, the commitment phase samples a mask vector $\boldsymbol{y}$ consisting of polynomials with small coefficients from the set $\widetilde{S}_{\gamma_1} \coloneqq \{w \in R \mid w = w \bmod^{\pm} 2\gamma_1\}$. After this, the commitment $\mathsf{HighBits}(\boldsymbol{A}\boldsymbol{y}, 2\gamma_2)$, combined with the message hash, is mapped via a hash function $\mathsf{H}\colon \{0,1\}^* \to B_\tau$ to a challenge polynomial c in $B_\tau \coloneqq \{w \in R \mid \|w\|_\infty = 1 \text{ and } \|w\|_1 = \tau\}$. Using rejection sampling, this process is repeated until the signature $(c, \boldsymbol{z} = c\,\boldsymbol{s}_1 + \boldsymbol{y})$ reveals no information about the secret vectors $\boldsymbol{s}_1$ and $\boldsymbol{s}_2$, and the publicly computable value $\mathsf{HighBits}(\boldsymbol{A}\boldsymbol{z} - c\,\boldsymbol{t}, 2\gamma_2)$, calculated with the public vector $\boldsymbol{t} = \boldsymbol{A}\boldsymbol{s}_1 + \boldsymbol{s}_2$, agrees with the commitment. To verify the signature, one checks that the entries of $\boldsymbol{z}$ have small coefficients and that the message hash, combined with the publicly computable commitment, maps under the hash function H to the challenge polynomial c.

Algorithm 1 ML-DSA key generation (simplified version)

Output: Public key pk and private key sk.

1: $\boldsymbol{A} \leftarrow_{\$} R_q^{k\times\ell}$
2: $(\boldsymbol{s}_1, \boldsymbol{s}_2) \leftarrow_{\$} S_\eta^\ell \times S_\eta^k$
3: $\boldsymbol{t} \leftarrow \boldsymbol{A}\boldsymbol{s}_1 + \boldsymbol{s}_2$
4: **return** $\mathsf{pk} \leftarrow (\boldsymbol{A}, \boldsymbol{t})$, $\mathsf{sk} \leftarrow (\boldsymbol{A}, \boldsymbol{s}_1, \boldsymbol{s}_2)$

Algorithm 2 ML-DSA signature generation (simplified version)

Input: Private key $\mathsf{sk} = (\boldsymbol{A}, \boldsymbol{s}_1, \boldsymbol{s}_2)$ and message hash $\mu \in \{0,1\}^{512}$.
Output: Signature $\sigma = (c, \boldsymbol{z})$.

1: **repeat**
2: $\quad \boldsymbol{y} \leftarrow_{\$} \widetilde{S}_{\gamma_1}^\ell$
3: $\quad c \leftarrow \mathsf{H}\big(\mu \parallel \mathsf{HighBits}(\boldsymbol{A}\boldsymbol{y}, 2\gamma_2)\big)$
4: $\quad \boldsymbol{z} \leftarrow c\,\boldsymbol{s}_1 + \boldsymbol{y}$
5: **until** $\|\boldsymbol{z}\|_\infty < \gamma_1 - \beta$ **and** $\|\mathsf{LowBits}(\boldsymbol{A}\boldsymbol{y} - c\,\boldsymbol{s}_2, 2\gamma_2)\|_\infty < \gamma_2 - \beta$
6: **return** $\sigma \leftarrow (c, \boldsymbol{z})$

Algorithm 3 ML-DSA signature verification (simplified version)

Input: Public key $\mathsf{pk} = (\boldsymbol{A}, \boldsymbol{t})$, message hash $\mu \in \{0,1\}^{512}$, and signature $\sigma = (c, \boldsymbol{z})$.
Output: **True** or **false**.

1: $c' \leftarrow \mathsf{H}\big(\mu \parallel \mathsf{HighBits}(\boldsymbol{A}\boldsymbol{z} - c\,\boldsymbol{t}, 2\gamma_2)\big)$
2: **return** $\|\boldsymbol{z}\|_\infty < \gamma_1 - \beta$ **and** $c = c'$

The security of ML-DSA relies on the assumed hardness of MLWE (module learning with errors) and a variant of MSIS (module short integer solution) over R_q to guarantee that signatures constitute proofs of knowledge and that secret vectors equivalent to those of the private key $(\boldsymbol{A}, \boldsymbol{s}_1, \boldsymbol{s}_2)$ cannot be feasibly recovered from the public key $(\boldsymbol{A}, \boldsymbol{t})$.

As the parameter γ_2 plays no further role in this paper, we denote γ_1 simply by γ.

2.3 Bit representation

The following function assigns bits to arbitrary (signed, arbitrary precision) integers.

Definition 2. For $j \geq 0$, we define the function

$$\mathrm{bit}_j\colon \mathbb{Z} \to \{0,1\}\,, \quad y \mapsto \lfloor y/2^j \rfloor \bmod 2\,.$$

We also set $\mathrm{bit}_{-1}(y) \coloneqq 0$ for all $y \in \mathbb{Z}$.

Remark 3. Let $y \in \mathbb{Z}$ and let $y_j = \mathrm{bit}_j(y)$ for $j \geq 0$. The representation of y as a 2-adic integer is $y = (\ldots y_2 y_1 y_0)_2$. For instance, we have $-1 = (1^\infty)_2$, $0 = (0^\infty)_2$, $1 = (0^\infty 1)_2$. Since $y + \overline{y} = (1^\infty)_2 = -1$, where $\overline{y}$ is the bitwise complement of y, we have $-y = \overline{y} + 1$.

Remark 4. Let $y \in [-2^{w-1}, 2^{w-1}) \cap \mathbb{Z}$ for some $w \geq 1$ and let $y_j = \mathrm{bit}_j(y)$ for $j \geq 0$. Then the w-bit two's complement representation of y is $(y_{w-1} y_{w-2} \ldots y_0)$, which is the w-bit truncation of the 2-adic integer $(y_{w-1}^\infty y_{w-2} \ldots y_0)_2$, and $y = \sum_{j=0}^{w-2} y_j 2^j - y_{w-1} 2^{w-1}$. In particular, for any $y \in \mathbb{Z}$, we have $y \bmod^\mp 2^w = \sum_{j=0}^{w-2} y_j 2^j - y_{w-1} 2^{w-1}$.

Remark 5. As noted in Remark 4, working with the function bit_j corresponds to using the two's complement representation, which is also the convention adopted in [Dam+25a; Dam+26; Sch+26]. In contrast, [Liu+21; Wan+21] use the sign-magnitude representation. For the leakage models studied, the different representations are not especially consequential, since the signs of the integers whose bits are leaked can be deduced with high probability, and, given the sign, the j-th bit in the sign-magnitude representation is highly correlated with the j-th bit in the two's complement representation. More precisely, let $w \geq 1$ and, for $y \in (-2^{w-1}, 2^{w-1}) \cap \mathbb{Z}$, let $(\widehat{y}_{w-1} \widehat{y}_{w-2} \ldots \widehat{y}_0)$ be its w-bit sign-magnitude representation, i.e. $y = (-1)^{\widehat{y}_{w-1}} \sum_{j=0}^{w-2} \widehat{y}_j 2^j$, and let $y_j = \mathrm{bit}_j(y)$. Then, for $j \in \{0, \ldots, w-1\}$, we have $y_j = \widehat{y}_j$ for all positive y. For negative y, we have $y_{w-1} = \widehat{y}_{w-1}$ and $y_0 = 1 - \widehat{y}_0$, and, among all $y \in (-2^{w-1}, 0) \cap \mathbb{Z}$, the proportion for which $y_j = 1 - \widehat{y}_j$ holds is $(1 - 2^{-j}) \cdot 2^{w-1} / (2^{w-1} - 1)$ for $j \in \{1, \ldots, w-2\}$.

3 Single-bit mask-vector leakage in ML-DSA

For the remainder of this paper, we shall work with a fixed choice of ML-DSA parameters η, $\gamma = \gamma_1$, τ, and $\beta = \tau \cdot \eta$ (see Table 1). Moreover, we fix $\lambda \geq 0$, which will play the role of the index of a leaked bit, and $p_{\mathsf{err}} \in [0, \frac{1}{2})$, which will determine an error probability.

Remark 6. We note that most results of this paper hold for other sensible choices of parameters as well. For some results of our theoretical analysis, however, we take advantage of the fact that γ is a power of 2 and $\gamma > 2\beta$. We note that both conditions are satisfied for the official ML-DSA parameter sets [FIPS204] as well as for the Dilithium challenge and extended security parameters [Dilithium, Section 4].

In this section we first introduce the abstract model and the specific goals of our attacks. We then discuss how our attack model applies to ML-DSA.

3.1 Problem statement

In this subsection, we introduce a reduced version of the ML-DSA signature generation algorithm with noisy single-bit mask leak.

Let $\mathcal{X} := ([-\eta, \eta] \cap \mathbb{Z})^{256}$ be the set of possible keys. Given a key $x^* \in \mathcal{X}$, Algorithm 4 computes a reduced signature $(c, z = \langle c, x^* \rangle + y) \in \mathcal{C} \times \mathcal{Z}$ using rejection sampling, where $\mathcal{C} := \{c \in \mathbb{Z}^{256} \mid \|c\|_\infty = 1 \text{ and } \|c\|_1 = \tau\}$ and $\mathcal{Z} := (-\gamma + \beta, \gamma - \beta) \cap \mathbb{Z}$. Here, $\langle c, x^* \rangle := \sum_{j=1}^{n} c_j x_j^*$ denotes the standard scalar product. Moreover, Algorithm 4 outputs a noisy bit leak $\widetilde{y}_\lambda = y_\lambda \oplus e \in \{0, 1\}$, where $y_\lambda := \mathrm{bit}_\lambda(y)$ is the mask bit of index $\lambda \geq 0$ according to Definition 2 and $e \in \{0, 1\}$ is an error bit that is 1 with probability $p_{\mathsf{err}} \in [0, \frac{1}{2})$, i.e. e is drawn from a Bernoulli distribution with parameter p_{err}. We refer to the combined output $(c, z, \widetilde{y}_\lambda)$ as *leaky signature*.

We say that we have *exact leakage* if $p_{\mathsf{err}} = 0$ (in this case, we have $e = 0$ and $\widetilde{y}_\lambda = y_\lambda$) and that we have *noisy leakage* if $p_{\mathsf{err}} > 0$. We may assume that p_{err} is known. However, we do not require this knowledge for our attacks.

The main object of investigation in this paper is to study the following attack model; see [Dam+25a, Definition 1].

Algorithm 4 Reduced ML-DSA signature generation with noisy single-bit leak

Input: A key $x^* \in \mathcal{X}$.

Output: A leaky signature $(c, z, \widetilde{y}_\lambda) \in \mathcal{C} \times \mathcal{Z} \times \{0,1\}$ such that $\widetilde{y}_\lambda = \mathrm{bit}_\lambda(z - \langle c, x^* \rangle)$ holds with probability $1 - p_{\mathsf{err}}$.

1: **repeat**
2: $\quad y \leftarrow_{\$} (-\gamma, \gamma] \cap \mathbb{Z}$
3: $\quad c \leftarrow_{\$} \mathcal{C}$
4: $\quad z \leftarrow \langle c, x^* \rangle + y$
5: **until** $|z| < \gamma - \beta$
6: $y_\lambda \leftarrow \mathrm{bit}_\lambda(y)$
7: $e \leftarrow_{\$} \mathsf{Ber}(p_{\mathsf{err}})$
8: $\widetilde{y}_\lambda \leftarrow y_\lambda \oplus e$
9: **return** $(c, z, \widetilde{y}_\lambda)$

Problem 1. *Let x^* denote a key, drawn uniformly at random from $\mathcal{X}$ and unknown to the attacker. The attacker is permitted to repeatedly request leaky signatures generated by independent executions of Algorithm 4 with input x^*. Eventually, the attacker has to present a key guess $\widehat{x} \in \mathcal{X}$ and the attack is successful if $\widehat{x} = x^*$.*

The number of requested leaky signatures is denoted by n_{sig} and the transcript of leaky signatures is denoted by $(c^{(i)}, z^{(i)}, \widetilde{y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}}$.

Remark 7. The leakage model in [Liu+21; Wan+21] is obtained by replacing, in line 6 of Algorithm 4, $\mathrm{bit}_\lambda(y)$ with the λ-th bit $\widehat{y}_\lambda$ of the sign-magnitude representation of y. For the discussion here, let us assume $0 < \lambda < \log_2(\gamma)$, since otherwise $\mathrm{bit}_\lambda(y)$ and $\widehat{y}_\lambda$ are deterministically related to each other anyway (see Remark 5). Given an output $(c, z, \overline{y}_\lambda)$ of this modified algorithm, define $\widetilde{\overline{y}}_\lambda = \overline{y}_\lambda$ when $z \geq 0$, and $\widetilde{\overline{y}}_\lambda = 1 - \overline{y}_\lambda$ when $z < 0$. Since z and y have the same sign with high probability, the discussion in Remark 5 shows that the distribution of samples $(c, z, \widetilde{\overline{y}}_\lambda)$ produced in this way is close to the output distribution of the unmodified Algorithm 4, but with an increased error probability of roughly $2^{-\lambda-1} + p_{\mathsf{err}}(1 - 2^{-\lambda})$.

3.2 Connection to ML-DSA

Let $\mathsf{sk} = (\boldsymbol{A}, \boldsymbol{s}_1, \boldsymbol{s}_2)$ be a private key and $\mathsf{pk} = (\boldsymbol{A}, \boldsymbol{t})$ the corresponding public key output by Algorithm 1. In this simplified version of ML-DSA key generation, finding the secret component $\boldsymbol{s}_1$ is sufficient to fully reconstruct sk. Indeed, the remaining secret component satisfies $\boldsymbol{s}_2 = \boldsymbol{t} - \boldsymbol{A}\boldsymbol{s}_1$ and can be computed from $\boldsymbol{s}_1$ and the public key pk.

Remark 8. For ML-DSA as specified in [FIPS204], i.e., without our simplifications, knowledge of $\boldsymbol{s}_1$ still suffices to derive signing data $(\boldsymbol{A}, \boldsymbol{s}_1, \boldsymbol{s}_2')$, which can be used to forge signatures that verify under the public data $(\boldsymbol{A}, \boldsymbol{t}_1)$, where $\boldsymbol{t}_1$ are the high-order bits of $\boldsymbol{t}$; see, e.g., [BP18, Section 4] or [Rav+18]. Even though such an $\boldsymbol{s}_2'$ is straightforward to obtain, it is not feasible to determine the true secret component $\boldsymbol{s}_2$ from $\boldsymbol{s}_1$ and $(\boldsymbol{A}, \boldsymbol{t}_1)$ alone. However, with the additional knowledge of a sufficiently large number of signatures, the low-order bits $\boldsymbol{t}_0$ of $\boldsymbol{t}$ can be reconstructed from $(\boldsymbol{A}, \boldsymbol{t}_1)$; see [Oli+25]. In this situation, $\boldsymbol{s}_2$ can be computed from $(\boldsymbol{A}, \boldsymbol{t} = 2^{13}\,\boldsymbol{t}_1 + \boldsymbol{t}_0)$ exactly as in our simplified version.

Thus, in practice, recovering the private key sk, reduces to determining the ℓ components of the vector $\boldsymbol{s}_1$ one by one. In theory, knowing even a single component of $\boldsymbol{s}_1$, already substantially reduces the computation required to recover the entire vector; see [Dam+25a, Section 4.3]. From now on, we therefore focus on the problem of recovering a single fixed coordinate $\boldsymbol{s}_{1,i}$. To this end, applying the i-th coordinate projection $R^\ell \to R$ to the vector

equation $\boldsymbol{z} = c\,\boldsymbol{s}_1 + \boldsymbol{y}$ corresponding to line 4 of Algorithm 2 yields the polynomial identity

$$\boldsymbol{z}_i = c\,\boldsymbol{s}_{1,i} + \boldsymbol{y}_i\,.$$

With the $\mathbb{Z}$-module isomorphism $\mathrm{coeff}\colon R \to \mathbb{Z}^{256}$ that sends polynomials $v = \sum_{j=1}^{256} v_j X^{j-1}$ to their coefficient vectors $(v_1, \ldots, v_{256})$, we can rewrite this as the matrix equation

$$\mathrm{coeff}(\boldsymbol{z}_i) = \mathrm{mul}(c)\,\mathrm{coeff}(\boldsymbol{s}_{1,i}) + \mathrm{coeff}(\boldsymbol{y}_i)\,,$$

where $\mathrm{mul}(c)$ is the $\mathbb{Z}$-module endomorphism of $\mathbb{Z}^{256}$ satisfying $\mathrm{coeff}(cv) = \mathrm{mul}(c)\,\mathrm{coeff}(v)$ for all $v \in R$. Due to the identity $X^{256} = -1$ in R, the coefficients of the product cv are $(cv)_j = \sum_{j'=1}^{j} c_{j-j'+1} v_{j'} + \sum_{j'=j+1}^{256} (-c_{256+j-j'+1}) v_{j'}$, and therefore the matrix $\mathrm{mul}(c)$, expressed in the standard basis, has as its j-th row the vector

$$\mathrm{mul}(c)_j = (c_j, c_{j-1}, \ldots, c_1, -c_{256}, -c_{255}, \ldots, -c_{j+1})\,.$$

Applying the j-th coordinate projection $\mathbb{Z}^{256} \to \mathbb{Z}$ to the matrix equation above gives

$$\mathrm{coeff}(\boldsymbol{z}_i)_j = \langle \mathrm{mul}(c)_j, \mathrm{coeff}(\boldsymbol{s}_{1,i}) \rangle + \mathrm{coeff}(\boldsymbol{y}_i)_j\,.$$

After redefining c as $\mathrm{mul}(c)_j$ and setting $x^* = \mathrm{coeff}(\boldsymbol{s}_{1,i})$, $y = \mathrm{coeff}(\boldsymbol{y}_i)_j$, and $z = \mathrm{coeff}(\boldsymbol{z}_i)_j$, this equation takes the form $z = \langle c, x^* \rangle + y$, which matches line 4 of Algorithm 4.

Remark 9. We adopt the convention that c, $\boldsymbol{s}_1$, $\boldsymbol{y}$, $c\boldsymbol{s}_1$, and $c\boldsymbol{s}_1 + \boldsymbol{y}$ have coefficients in $\mathbb{Z}$. Since these coefficients all lie in the interval $\left(-\frac{q}{2}, \frac{q}{2}\right)$, viewing the identity $\boldsymbol{z} = c\boldsymbol{s}_1 + \boldsymbol{y}$ over $\mathbb{Z}_q$ is equivalent to interpreting it as an identity over $\mathbb{Z}$. Consequently, in addressing Problem 1, all arithmetic operations may be carried out over $\mathbb{Z}$ rather than $\mathbb{Z}_q$. Computations over $\mathbb{Z}_q$ are required only in those parts of Algorithm 1, Algorithm 2, and Algorithm 3 where the matrix $\boldsymbol{A}$ appears. In Algorithm 4 all operations are performed over $\mathbb{Z}$.

Remark 10. The map coeff restricts to a bijection $\widetilde{S}_\gamma \to ((-\gamma, \gamma] \cap \mathbb{Z})^{256}$. Consequently, deriving y from $\boldsymbol{y}$ as described above, where $\boldsymbol{y}$ is sampled according to line 2 of Algorithm 2, yields a uniform distribution, just as sampling y directly in line 2 of Algorithm 4.

Remark 11. For every $j \in [256]$, the rule $c \mapsto \mathrm{mul}(c)_j$ defines a bijection $B_\tau \to \mathcal{C}$. Therefore, if c is sampled uniformly from B_τ, the induced distribution of $\mathrm{mul}(c)_j$ is uniform on $\mathcal{C}$, matching the distribution of the variable sampled in line 3 of Algorithm 4. In fact, ML-DSA assumes that the distribution of the challenge polynomial c in line 3 of Algorithm 2 is close to uniform on B_τ. When H is treated as an idealized hash function (e.g., a random oracle), this assumption is reasonable because collisions in the mapping $\boldsymbol{y} \mapsto \mathsf{HighBits}(\boldsymbol{A}\boldsymbol{y}, 2\gamma_2)$, for uniformly sampled $\boldsymbol{y}$, are extremely unlikely and the mapping's range is vastly larger than the range B_τ of H.

Remark 12. Algorithm 4 assumes that y and c are sampled independently in lines 2 and 3. In contrast, in lines 2 and 3 of Algorithm 2, the challenge polynomial c is fully determined by the mask vector $\boldsymbol{y}$. Nevertheless, for fixed indices i and j and for each $y \in (-\gamma, \gamma] \cap \mathbb{Z}$, the mapping $\boldsymbol{y} \mapsto \mathsf{HighBits}(\boldsymbol{A}\boldsymbol{y}, 2\gamma_2)$, when restricted to vectors $\boldsymbol{y}$ satisfying $\mathrm{coeff}(\boldsymbol{y}_i)_j = y$, still produces collisions only with negligible probability, and its range remains vastly larger than the range of H. Modeling H as an idealized hash function therefore justifies treating c as nearly independent of $\mathrm{coeff}(\boldsymbol{y}_i)_j$.

Remark 13. In line 5 of Algorithm 4, (c, z) is accepted whenever $|z| < \gamma - \beta$. In contrast, in line 5 of Algorithm 2, $(c, \mathrm{coeff}(\boldsymbol{z}_i)_j)$ is accepted only if additional conditions are satisfied. However, conditioning on the event $|\mathrm{coeff}(\boldsymbol{z}_{i'})_{j'}| < \gamma - \beta$ for all $(i', j') \neq (i, j)$ should not significantly affect the distribution of $(c, \mathrm{coeff}(\boldsymbol{z}_i)_j)$ conditioned on $|\mathrm{coeff}(\boldsymbol{z}_i)_j| < \gamma - \beta$. It also is plausible that further conditioning on $\|\mathsf{LowBits}(\boldsymbol{A}\boldsymbol{y} - c\,\boldsymbol{s}_2, 2\gamma_2)\|_\infty < \gamma_2 - \beta$ has little effect on this distribution, owing to the unstructured nature of $\boldsymbol{A}$ and the uniform sampling of $\boldsymbol{y}$. Note that ML-DSA, without our simplifications, includes a third acceptance

condition; see line 28 of [FIPS204, Algorithm 7], which also appears unlikely to substantially alter the distribution. In particular, its first part, $\|c\boldsymbol{t}_0\|_\infty < \gamma_2$, is non-vacuous only for ML-DSA-44, where the expected probability of violation is small (below 7.38×10^{-9}).

While these assumptions appear justified, the exact distribution of z given $|z| < \gamma - \beta$ is not essential to our attacks, as long as the probability of z being *informative* (see Section A.3) is sufficiently high. Nonetheless, for our analysis, particularly for determining the expected number of signatures required for the attacks to succeed, we rely on (c, z) being sampled according to Algorithm 4.

To summarize, for $i \in [\ell]$ and $j \in [256]$, the tuples $(\mathrm{mul}(c)_j, \mathrm{coeff}(\boldsymbol{z}_i)_j, \mathrm{bit}_\lambda(\mathrm{coeff}(\boldsymbol{y}_i)_j))$, where $(c, \boldsymbol{z})$ denote the outputs and $\boldsymbol{y}$ the values in the final loop iteration of Algorithm 2 under input $\mathsf{sk} = (\boldsymbol{A}, \boldsymbol{s}_1, \boldsymbol{s}_2)$, follow essentially the same distribution as the outputs of Algorithm 4 under input $x^* = \mathrm{coeff}(\boldsymbol{s}_{1,i})$ in the situation of exact leakage (i.e. $p_{\mathsf{err}} = 0$). The discussion extends to the situation of noisy leakage with obvious modifications.

4 Optimal key recovery and required number of signatures

Let $x^* \in \mathcal{X}$ be a key and let $(c^{(i)}, z^{(i)}, \widetilde{y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}}$ be leaky signatures generated by independent runs of Algorithm 4. The goal of Problem 1 is to find a key guess $\widehat{x} \in \mathcal{X}$ that equals the true key x^* with high probability. An essential step in assessing the complexity of this task is estimating the minimal number n_{sig} of signatures required so that an optimal key guess $\widehat{x}$ typically is equal to the true key x^*. In this section, we derive such estimates.

As a first step, in Section 4.1, we introduce the sets of *candidate keys* and *most likely keys*. These sets represent, in the case of exact or noisy leakage, respectively, the sets of optimal key guesses for Problem 1 for a given number of signatures. Then, in Section 4.2, we introduce the notion of *informativeness*. This notion helps us to filter the set of leaky signatures for those that provide information about the key.

In Section 4.3, we then, in the *high-λ regime* (i.e. if $\lambda > \log_2(\beta) + 1$) and in the case of *exact leakage*, provide rigorous bounds for the required number of signatures. In Section 4.4 we generalize one of these bounds to the case of *noisy leakage*. Later, in Section A.4 and Section A.5, respectively, we show how these rigorous bounds can be extended via experimentally backed heuristic arguments to the *low-λ regime* (i.e. if $\lambda < \log_2(\beta) + 1$) and, in the noisy leakage case, to also provide the missing bound.

Formal definitions and proofs for the results in this section are deferred to Appendix A.

4.1 Candidate keys and most likely keys

Let $x^* \in \mathcal{X}$ be a key and let $(c^{(i)}, z^{(i)}, \widetilde{y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}}$ be leaky signatures generated by independent runs of Algorithm 4.

An element $x \in \mathcal{X}$ is said to satisfy the *bit constraint* imposed by a leaky signature $(c, z, \widetilde{y}_\lambda)$ if $\widetilde{y}_\lambda = \mathrm{bit}_\lambda(z - \langle c, x\rangle)$ (regardless of whether $\widetilde{y}_\lambda$ is the correct bit). In the case of exact leakage ($p_{\mathsf{err}} = 0$), the true key x^* satisfies all bit constraints. Therefore, we define the set of *candidate keys* by

$$\mathcal{X}_{\mathsf{cand}} = \left\{x \in \mathcal{X} \mid \widetilde{y}_\lambda{}^{(i)} = \mathrm{bit}_\lambda(z^{(i)} - \langle c^{(i)}, x\rangle) \text{ for all } i \in [n_{\mathsf{sig}}]\right\}.$$

In the general case ($p_{\mathsf{err}} \in [0, \frac{1}{2})$), the best we can hope for is to find keys for which the number of violated bit constraints is minimized. Accordingly, we define the set of *most likely keys* by

$$\mathcal{X}_{\mathsf{ml}} = \operatorname*{arg\,min}_{x \in \mathcal{X}} \#\left\{i \in [n_{\mathsf{sig}}] \mid \widetilde{y}_\lambda{}^{(i)} \neq \mathrm{bit}_\lambda(z^{(i)} - \langle c^{(i)}, x\rangle)\right\}.$$

where $\arg\min$ denotes the set of all $x \in \mathcal{X}$ for which the minimum is attained. Note that $\mathcal{X}_{\mathsf{cand}}$ and $\mathcal{X}_{\mathsf{ml}}$ coincide whenever $\mathcal{X}_{\mathsf{cand}}$ is non-empty.

Optimal key guesses

Intuitively, the set $\mathcal{X}_{\mathsf{ml}}$ should contain good key guesses $\widehat{x}$ to solve Problem 1. Based on the stochastic model in Section A.1, Corollary 42 in Section A.2 confirms that picking key guesses from $\mathcal{X}_{\mathsf{ml}}$ is indeed optimal in the sense that the success probability of solving Problem 1 is maximized.

4.2 Informative signatures

Recall that the goal of Problem 1 is, given a family of leaky signatures, to compute a key guess that equals the true key x^*. Reaching this goal crucially depends on the amount of information this family of leaky signatures contains about the key. In information theory, such a quantification of information is typically done via the notion of *mutual information*.

In Section A.3 we consider a variant of mutual information that quantifies the amount of information a given leaky signature $(c, z, \widetilde{y}_\lambda)$ contains about the random and unknown key x^*. This notion can be considered to be a mixture of pointwise mutual information w.r.t. $(c, z, \widetilde{y}_\lambda)$ and standard mutual information w.r.t. x^*.

If this mutual information is strictly positive, we say that $(c, z, \widetilde{y}_\lambda)$ is *informative*. This definition basically coincides with the notion of the same name introduced in [Dam+25a]. By Lemma 48 (c) in Section A.3, we have the characterization

$$(c, z, \widetilde{y}_\lambda) \text{ is informative} \qquad \text{if and only if} \qquad z \in [-\beta, \beta) + 2^\lambda \mathbb{Z}\,.$$

In particular, the informativeness of $(c, z, \widetilde{y}_\lambda)$ only depends on the public signature part z.

Restriction to informative signatures

If a leaky signature is uninformative, its bit constraint is satisfied either by all keys or by no key. Therefore, uninformative signatures can be filtered out without changing the sets $\mathcal{X}_{\mathsf{cand}}$ and $\mathcal{X}_{\mathsf{ml}}$ defined in Section 4.1. The sets of candidate keys and most likely keys given n_{inf} *informative* signatures will be denoted by $\mathcal{X}_{\mathsf{cand,inf}}$ and $\mathcal{X}_{\mathsf{ml,inf}}$, respectively.

We note that the notion of mutual information defined in Section A.3 would also enable us to filter out leaky signatures with a small (but positive) amount of information about the key. However, since storage and processing of all informative signatures does not pose a bottleneck for our attack experiments in Section 5 and Section 6, we do not use any additional filtering in this paper.

Proportion of informative signatures

Since the data complexity of Problem 1 is determined by the (unrestricted) number of leaky signatures, we are interested in the proportion of informative signatures. By Lemma 49 (b), the probability that a random leaky signature is informative is

$$p_{\mathsf{inf}} = \begin{cases} 1 & \text{if } 0 \le \lambda \le \log_2(\beta) + 1\,, \\ \frac{2\beta(\gamma/2^{\lambda-1} - 1)}{2(\gamma-\beta)-1} & \text{if } \log_2(\beta) + 1 < \lambda \le \log_2(\gamma)\,, \\ \frac{2\beta}{2(\gamma-\beta)-1} & \text{if } \lambda \ge \log_2(\gamma)\,. \end{cases}$$

Figure 3 illustrates p_{inf} on a logarithmic scale for various sets of parameters and bit positions. We will refer to parameters with $\lambda < \log_2(\beta) + 1$ as the *low-λ regime* and to parameters with $\lambda > \log_2(\beta) + 1$ as the *high-λ regime*.

In the low-λ regime, every leaky signature is informative. For $\log_2(\beta)+1 < \lambda \ll \log_2(\gamma)$, we have $p_{\mathsf{inf}} \approx \beta/2^{\lambda-1}$ in good approximation, hence the proportion of informative signatures is roughly halved for consecutive bit positions in this regime.

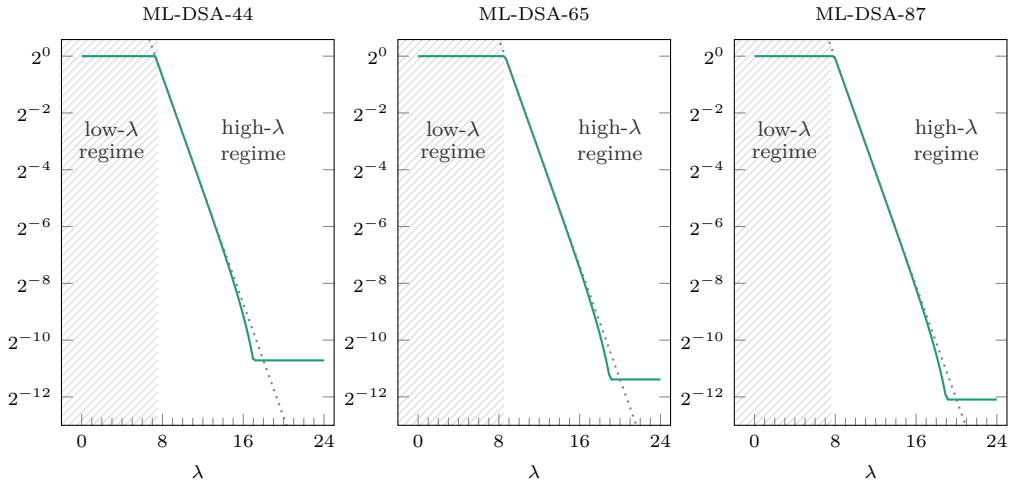


Figure 3: Logarithmic plots of the probability p_{inf} that a leaky signature is informative (solid lines) and the approximation $\beta/2^{\lambda-1}$ (dotted lines).

By Lemma 53, n_{sig} leaky signatures contain $n_{\mathsf{sig}} \cdot p_{\mathsf{inf}}$ informative signatures on average. Conversely, $n_{\mathsf{inf}}/p_{\mathsf{inf}}$ leaky signatures have to be generated on average to obtain n_{inf} informative signatures. This makes it possible to convert the required number of informative signatures into the required number of unrestricted leaky signatures in expectation.

Remark 14. When full ML-DSA is considered, the informativeness only depends on the coefficient of the component of $\boldsymbol{z}$ corresponding to the leaked bit (see the reduction from ML-DSA signatures with bit leak to leaky signatures of Problem 1 in Section 3.2). The other coefficients of $\boldsymbol{z}$ have no influence on the informativeness of the extracted leaky signature.

Uniform hardness in the high-λ regime

Given the same number of informative signatures, random instances of Problem 1 are equally hard to solve for all choices of λ in the high-λ regime, see Lemma 54 and Lemma 55. This justifies focusing henceforth on $\lambda = 8$ for ML-DSA-44 and ML-DSA-87, and on $\lambda = 9$ for ML-DSA-65, as the representative choices for λ in the high-λ regime.

4.3 Expected number of candidate keys

In this subsection we provide estimates on the required number of leaky signatures in our attacks. More precisely, in the case of exact leakage, we aim to find the number n_{inf} of informative signatures such that the expected number of candidate keys, $\mathbb{E}(\#\mathfrak{X}_{\mathsf{cand,inf}})$, is close to 1.

In the following theorem, a combination of Lemma 60 and Lemma 61 from Section A.4, we state upper and lower bounds on this expected value, from which we can afterwards deduce the desired number of signatures. The result will be formulated for informative signatures, i.e. in terms of n_{inf}. As explained in Remark 62, the results can easily be transferred to n_{sig}, i.e. without the restriction to the informativeness of the signatures. The reason we chose n_{inf} instead of n_{sig} are uniformness properties of informativeness in the high-λ regime; see Lemma 54 and Lemma 55.

Theorem 15. *Assuming* $\lambda > \log_2(\beta) + 1$ *and* $p_{\mathsf{err}} = 0$*, we have*

$$\Theta_{\mathsf{lower}}(n_{\mathsf{inf}}) \leq \mathbb{E}(\#\mathfrak{X}_{\mathsf{cand,inf}}) \leq \Theta_{\mathsf{upper}}(n_{\mathsf{inf}})\,,$$

where the functions $\Theta_{\mathsf{lower}}(n_{\mathsf{inf}})$ *and* $\Theta_{\mathsf{upper}}(n_{\mathsf{inf}})$ *are defined in Section A.4.* □

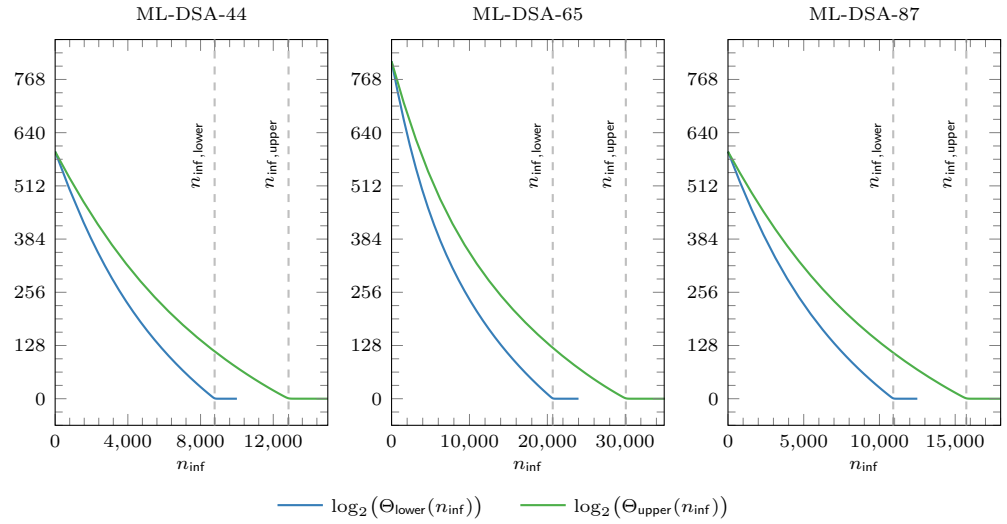


Figure 4: Binary logarithm of the proven bounds for $\mathbb{E}(\#\mathfrak{X}_{\mathsf{cand,inf}})$.

We now deduce from this result a useful estimate on the expected number of signatures required to recover a unique key, which holds for all bit positions in the high-λ regime.

As can be seen in Section A.4, the bounds $\Theta_{\mathsf{lower}}(n_{\mathsf{inf}})$ and $\Theta_{\mathsf{upper}}(n_{\mathsf{inf}})$ are monotonically decreasing and converge to 1 as n_{inf} approaches ∞, although they remain strictly greater than 1 for every finite n_{inf}. We define, in the high-λ regime,

$$n_{\mathsf{inf,lower}} := \min\{n_{\mathsf{inf}} \in \mathbb{N} \mid \Theta_{\mathsf{lower}}(n_{\mathsf{inf}}) < 2\}\,,$$
$$n_{\mathsf{inf,upper}} := \min\{n_{\mathsf{inf}} \in \mathbb{N} \mid \Theta_{\mathsf{upper}}(n_{\mathsf{inf}}) < 2\}\,.$$

Their values can be extracted from Figure 4 and are listed in the following table:

	ML-DSA-44	ML-DSA-65	ML-DSA-87
$n_{\mathsf{inf,lower}}$	8 774	20 692	10 909
$n_{\mathsf{inf,upper}}$	12 841	30 076	15 734

Note that, for $\Theta = \Theta_{\mathsf{lower}}$ or $\Theta = \Theta_{\mathsf{upper}}$, the plots show that even when replacing the condition $\Theta(n_{\mathsf{inf}}) < 2$ with $\Theta(n_{\mathsf{inf}}) < 1 + \varepsilon$ for some ε moderately above 0, the resulting values for the bound on n_{inf} would increase only modestly.

Remark 16. According to Theorem 15, in the high-λ regime and assuming exact leakage, recovering a unique key that could have produced the observed signatures should typically require at least $n_{\mathsf{inf,lower}}$ and not more than $n_{\mathsf{inf,upper}}$ informative leaky signatures. This is consistent with the experimental findings reported in Section 5 and Section 6. Notably, on average, the number of informative signatures required for key recovery exceeded $n_{\mathsf{inf,lower}}$ by only a modest margin, with successful recovery occurring only occasionally with fewer signatures. By contrast, the experiments indicate that the upper bound $n_{\mathsf{inf,upper}}$ is not tight; see Remark 62 for more details.

Heuristic extension to the low-λ regime

Under some heuristic arguments, outlined in Remark 64, we can extend Theorem 15 also to the low-λ regime. Our experimental results indicate that these heuristics are accurate when $\lambda \geq 4$ for ML-DSA-44 and when $\lambda \geq 5$ for ML-DSA-65 and ML-DSA-87; see Figure 1. Establishing rigorous bounds in the low-λ regime remains an open problem.

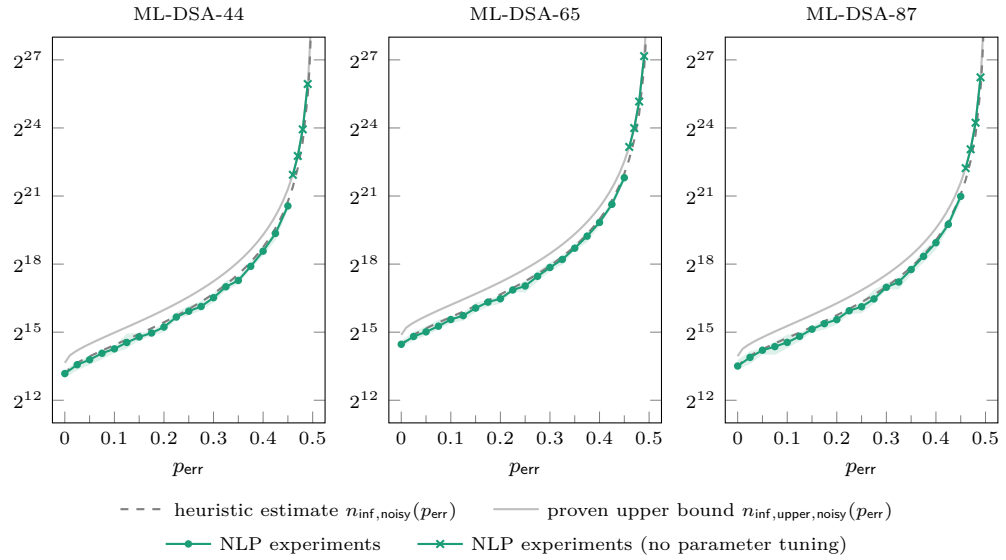


Figure 5: Estimates in the high-λ regime for the expected number of signatures required for key recovery, together with experimental results from Figure 11 for comparison.

4.4 Expected number of most likely keys

The last subsection focused on exact leakage, i.e. the case $p_{\mathsf{err}} = 0$. In the following theorem the results are partially extended to the noisy setting. Namely, we establish an upper bound on the expected number of most likely keys, $\mathbb{E}(\#\mathfrak{X}_{\mathsf{ml},\mathsf{inf}})$. Note that the theorem holds in both the exact and noisy setting. We defer its proof to Section A.5.

Theorem 17. *Assuming $\lambda > \log_2(\beta) + 1$, we have*

$$\mathbb{E}\left(\#\mathfrak{X}_{\mathsf{ml},\mathsf{inf}}\right) \leq \Theta_{\mathsf{upper},\mathsf{noisy}}(n_{\mathsf{inf}}, p_{\mathsf{err}}) \,,$$

where $\Theta_{\mathsf{upper},\mathsf{noisy}}$ is defined in Theorem 65. □

Number of signatures required to recover a unique key

We are again interested in the number of signatures required for our attacks. However, in this setting we are only given an upper bound. In Figure 5 we provide a plot of the corresponding quantity

$$n_{\mathsf{inf},\mathsf{upper},\mathsf{noisy}}(p_{\mathsf{err}}) \coloneqq \min\{n_{\mathsf{inf}} \in \mathbb{N} \mid \Theta_{\mathsf{upper},\mathsf{noisy}}(n_{\mathsf{inf}}, p_{\mathsf{err}}) < 2\} \,,$$

which seems to overestimate the values observed experimentally. However, building upon the rigorous result Theorem 65, in Remark 66 some heuristic arguments lead to the quantity

$$n_{\mathsf{inf},\mathsf{noisy}}(p_{\mathsf{err}}) \approx n_{\mathsf{inf},\mathsf{lower}} / \left(1 - 2\sqrt{(1-p_{\mathsf{err}})p_{\mathsf{err}}}\right)$$

as an estimate for the number of informative signatures required for our attacks (recall that we introduced $n_{\mathsf{inf},\mathsf{lower}}$ in Section 4.3). It can be seen in Figure 5 that this heuristic estimate seems to be close to the values observed experimentally.

5 Integer linear programming

In this section we establish integer linear program (ILP) formulations to compute candidate keys and most likely keys in the cases of exact and noisy leakage, respectively. We then report on experiments using several ILP solvers in practice.

5.1 ILP formulations

Fix a key $x^* \in \mathcal{X}$ and let $(c^{(i)}, z^{(i)}, \widetilde{y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}}$ be leaky signatures generated by independent runs of Algorithm 4.

Exact leakage

First we assume $p_{\mathsf{err}} = 0$. Let $\mathcal{X}_{\mathsf{cand}}$ be the set of candidate keys as defined in Section 4.1, i.e. $\mathcal{X}_{\mathsf{cand}} = \Xi_{\mathsf{cand}}\big((c^{(i)}, z^{(i)}, \widetilde{y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}}\big)$ according to Definition 39 (a).

The following lemma shows how to formulate the bit constraint imposed by a leaky signature as an ILP constraint. The lemma is similar to the transformation described in [Dam+25a, Section 4.4], but has a more streamlined description and proof. More importantly, the lemma is not restricted to the high-λ regime. Note that our notation is not compatible with [Dam+25a]; in particular, $\overline{z}$ is defined differently.

Lemma 18. *Let $x \in \mathcal{X}$ and let $(c, z, b) \in \mathcal{C} \times \mathbb{Z} \times \{0,1\}$ be such that $b = \mathrm{bit}_\lambda(z - \langle c, x\rangle)$. Set $\overline{z} \coloneqq z \bmod^{\mp} 2^\lambda \in [-2^{\lambda-1}, 2^{\lambda-1}) \cap \mathbb{Z}$ and $\kappa(z,b) \coloneqq b \oplus \mathrm{bit}_\lambda(z) \oplus \mathrm{bit}_{\lambda-1}(z) \oplus 1 \in \{0,1\}$.*

(a) There exists a unique $u \in \mathbb{Z}$ such that

$$\overline{z} - \kappa(z,b) \cdot 2^\lambda + 1 \le \langle c, x\rangle + u \cdot 2^{\lambda+1} \le \overline{z} - \kappa(z,b) \cdot 2^\lambda + 2^\lambda \,.$$

We have $u = \lfloor (\overline{z} - \langle c, x\rangle + 2^\lambda)/2^{\lambda+1} \rfloor \in [-\nu, \nu] \cap \mathbb{Z}$, where $\nu \coloneqq \lceil (\beta - 2^{\lambda-1})/2^{\lambda+1} \rceil$.

(b) If $\lambda \ge \log_2(\beta) + 1$, we have

$$\overline{z} + 1 - \kappa(z,b) \cdot (\overline{z} + 1 + \beta) \le \langle c, x\rangle \le \beta - \kappa(z,b) \cdot (\beta - \overline{z}) \,.$$

Proof. This follows directly from Lemma 44 with $s = \langle c, x\rangle$. □

We obtain the following integer linear program (feasibility problem) with $256 + n_{\mathsf{sig}}$ integer variables and n_{sig} interval constraints.

ILP 1. Find (x, u) subject to

$$\begin{aligned} \langle c^{(i)}, x\rangle + 2^{\lambda+1} u_i &\ge \overline{z}^{(i)} - \kappa(z^{(i)}, \widetilde{y}_\lambda{}^{(i)}) 2^\lambda + 1 && \text{for all } i \in [n_{\mathsf{sig}}] \,, \\ \langle c^{(i)}, x\rangle + 2^{\lambda+1} u_i &\le \overline{z}^{(i)} - \kappa(z^{(i)}, \widetilde{y}_\lambda{}^{(i)}) 2^\lambda + 2^\lambda && \text{for all } i \in [n_{\mathsf{sig}}] \,, \end{aligned}$$

where $x \in ([-\eta, \eta] \cap \mathbb{Z})^{256}$ and $u \in ([-\nu, \nu] \cap \mathbb{Z})^{n_{\mathsf{sig}}}$.

By Lemma 18, candidate keys $x \in \mathcal{X}_{\mathsf{cand}}$ are in one-to-one correspondence with feasible solutions (x, u) of ILP 1, where $u_i = \lfloor (\overline{z}^{(i)} - \langle c^{(i)}, x\rangle + 2^\lambda)/2^{\lambda+1} \rfloor$ for $i \in [n_{\mathsf{sig}}]$.

Remark 19. In the high-λ regime, we have $\nu = \lceil (\beta - 2^{\lambda-1})/2^{\lambda+1} \rceil = 0$ and the u-variables can be eliminated from ILP 1, leading to a program with a constant number of variables. In this case, we have $\mathcal{X}_{\mathsf{cand}} = \mathcal{P} \cap \mathbb{Z}^{256}$, where $\mathcal{P} \subset \mathbb{R}^{256}$ denotes the convex polytope given by the feasible region of the LP relaxation of ILP 1 (obtained by dropping the integrality constraints on x).

Remark 20. In the low-λ regime, where $\nu = \lceil (\beta - 2^{\lambda-1})/2^{\lambda+1} \rceil \ge 1$, it is possible to choose a smaller bound $0 \le \widetilde{\nu} < \nu$ for the u-variables in ILP 1, effectively pruning the feasible region of ILP 1 and eliminating the u-variables altogether if $\widetilde{\nu} = 0$, as long as the true key x^* remains in the feasible region with high probability. For ML-DSA-44, $\widetilde{\nu} = 1$ may be used for $\lambda = 4, 5$, and $\widetilde{\nu} = 0$ for $\lambda = 6, 7$. For ML-DSA-65, $\widetilde{\nu} = 1$ may be used for $\lambda = 5, 6, 7$, and $\widetilde{\nu} = 0$ for $\lambda = 8$. For ML-DSA-87, $\widetilde{\nu} = 2$ may be used for $\lambda = 4$, $\widetilde{\nu} = 1$ for $\lambda = 5, 6$, and $\widetilde{\nu} = 0$ for $\lambda = 7$.

Noisy leakage

Now we assume $p_{\mathsf{err}} \geq 0$. Let $\mathcal{X}_{\mathsf{ml}}$ be the set of most likely keys as defined in Section 4.1, i.e. $\mathcal{X}_{\mathsf{ml}} = \Xi_{\mathsf{ml}}\big((c^{(i)}, z^{(i)}, \widetilde{y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}}\big)$ according to Definition 39 (b).

Lemma 21. *Let* $(c, z, b) \in \mathcal{C} \times \mathbb{Z} \times \{0,1\}$. *Set* $\overline{z} \coloneqq z \bmod^{\mp} 2^\lambda \in [-2^{\lambda-1}, 2^{\lambda-1}) \cap \mathbb{Z}$ *and* $\kappa(z,b) \coloneqq b \oplus \mathrm{bit}_\lambda(z) \oplus \mathrm{bit}_{\lambda-1}(z) \oplus 1 \in \{0,1\}$. *Then, for every* $x \in \mathcal{X}$, *the inequalities*

$$\overline{z} - \kappa(z,b) \cdot 2^\lambda + 1 \leq \langle c, x\rangle + 2^{\lambda+1} \cdot u + (-1)^{\kappa(z,b)} 2^\lambda \cdot v \leq \overline{z} - \kappa(z,b) \cdot 2^\lambda + 2^\lambda$$

have a unique solution $(u, v) \in \mathbb{Z} \times \{0,1\}$ *given by*

$$u = \lfloor (\overline{z} - \langle c, x\rangle + 2^\lambda)/2^{\lambda+1} \rfloor \in [-\nu, \nu] \cap \mathbb{Z} \quad \textit{and} \quad v = 1_{b \neq \mathrm{bit}_\lambda(z - \langle c,x\rangle)}\,,$$

where $\nu \coloneqq \lceil (\beta - 2^{\lambda-1})/2^{\lambda+1} \rceil$.

Proof. Let $x \in \mathcal{X}$ and $y_\lambda \coloneqq \mathrm{bit}_\lambda(z - \langle c, x\rangle)$. We have $y_\lambda = b \oplus v$ with $v = 1_{b \neq \mathrm{bit}_\lambda(z - \langle c,x\rangle)}$. We obtain $\kappa(z, y_\lambda) = y_\lambda \oplus \mathrm{bit}_\lambda(z) \oplus \mathrm{bit}_{\lambda-1}(z) \oplus 1 = b \oplus v \oplus \mathrm{bit}_\lambda(z) \oplus \mathrm{bit}_{\lambda-1}(z) \oplus 1 = \kappa(z,b) + (-1)^{\kappa(z,b)} v$. Now the assertion follows from Lemma 18 applied to (c, z, y_λ). □

We obtain the following integer linear program (minimization problem) with $256 + n_{\mathsf{sig}}$ integer variables, n_{sig} boolean variables, and n_{sig} interval constraints.

ILP 2. Minimize $\sum_{i=1}^{n_{\mathsf{sig}}} v_i$ subject to

$$\langle c^{(i)}, x\rangle + 2^{\lambda+1} u_i + (-1)^{\kappa(z^{(i)}, \widetilde{y}_\lambda{}^{(i)})} 2^\lambda v_i \geq \overline{z}^{(i)} - \kappa(z^{(i)}, \widetilde{y}_\lambda{}^{(i)}) 2^\lambda + 1 \qquad \text{for all } i \in [n_{\mathsf{sig}}]\,,$$
$$\langle c^{(i)}, x\rangle + 2^{\lambda+1} u_i + (-1)^{\kappa(z^{(i)}, \widetilde{y}_\lambda{}^{(i)})} 2^\lambda v_i \leq \overline{z}^{(i)} - \kappa(z^{(i)}, \widetilde{y}_\lambda{}^{(i)}) 2^\lambda + 2^\lambda \qquad \text{for all } i \in [n_{\mathsf{sig}}]\,,$$

where $x \in ([-\eta, \eta] \cap \mathbb{Z})^{256}$, $u \in ([-\nu, \nu] \cap \mathbb{Z})^{n_{\mathsf{sig}}}$, and $v \in \{0,1\}^{n_{\mathsf{sig}}}$.

By Lemma 21, most likely keys $x \in \mathcal{X}_{\mathsf{ml}}$ are in one-to-one correspondence with optimal solutions (x, u, v) of ILP 2, where $u_i = \lfloor (\overline{z}^{(i)} - \langle c^{(i)}, x\rangle + 2^\lambda)/2^{\lambda+1} \rfloor$ and $v_i = 1_{\widetilde{y}_\lambda{}^{(i)} \neq \mathrm{bit}_\lambda(z^{(i)} - \langle c^{(i)}, x\rangle)}$ for $i \in [n_{\mathsf{sig}}]$.

Remark 22. The boolean variables v_i capture bit-constraint violations. If $p_{\mathsf{err}} = 0$, the minimum objective value is 0, obtained for $v = 0$, and ILP 2 becomes equivalent to ILP 1.

Remark 23. By Lemma 21, any $x \in \mathcal{X}$ can be easily lifted to a feasible solution (x, u, v) of ILP 2. This enables the development of tailored primal heuristics. For instance, given an optimal solution $(\widehat{x}, \widehat{u}, \widehat{v}) \in \mathbb{R}^{256+2n_{\mathsf{sig}}}$ of an LP relaxation of ILP 2, we may obtain $x = \lfloor \widehat{x} \rceil$ by rounding and lift it to a feasible solution, whereas the feasibility of $(\lfloor \widehat{x} \rceil, \lfloor \widehat{u} \rceil, \lfloor \widehat{v} \rceil)$ would not be guaranteed.

5.2 Experiments

In each experiment, we assume that the bit index λ and the error probability p_{err} are fixed, and informative leaky signatures $(c^{(i)}, z^{(i)}, \widetilde{y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{inf}}}$ are given that have been generated by Algorithm 4 with input x^*. To emphasize that only informative signatures are considered, we denote their count by n_{inf} rather than n_{sig}. By Lemma 53 (b), the expected number of leaky signatures required to obtain n_{inf} informative signatures is $\mathbb{E}(N_{\mathsf{sig}}) = n_{\mathsf{inf}}/p_{\mathsf{inf}}$, where p_{inf} is given by Lemma 49 (b).

We implemented the ILP attacks using the Julia programming language [Julia] and the JuMP modeling language [JuMP]. We conducted our experiments with the open-source ILP solvers HiGHS [HiGHS] and SCIP [SCIP], the commercial ILP solver Gurobi [Gurobi], as well as the open-source constraint programming solver CP-SAT [CP-SAT].

State-of-the-art ILP optimizers typically rely on presolving, a branch-and-cut implementation combining the branch-and-bound algorithm with cutting planes, as well as various

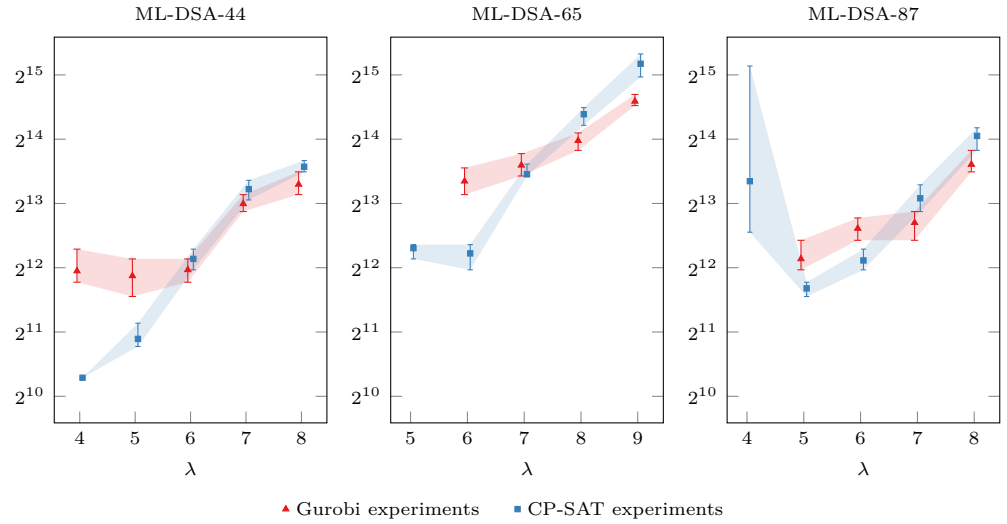


Figure 6: Number of informative signatures with exact leakage required for key recovery via solving ILP 1 using Gurobi's feasibility pump (4 cores) and CP-SAT (64 cores), each with a 10-minute time limit per experiment: minimum, maximum, and average value across 10 experiments per data point.

heuristics. In our experiments, we observed that ILP 1 with $\nu = 0$ (or $\widetilde{\nu} = 0$ as indicated in Remark 20) can be solved by all solvers using a moderate number of informative signatures, but ILP 1 with $\nu > 0$ and ILP 2 appear challenging for branch-and-cut implementations due to the additional variables u and v.

However, for ILP 1 we obtained good results by using Gurobi's feasibility pump heuristic (which is disabled by default) without the branch-and-cut procedure, which outperforms branch-and-cut implementations in terms of required signatures and also works for several parameters with $\nu > 0$. We obtained similar results with the constraint programming solver CP-SAT that seems to employ related heuristics. Therefore, we will only present results for Gurobi and CP-SAT below.

Figure 6 contains our experimental results for solving ILP 1 across various parameter sets and bit positions λ. In each parameter setting, we ran 10 experiments, gradually increasing the amount of informative signatures (in steps of 500) until the instance could be solved within a 10-minute time limit. Gurobi was running on a laptop with 4 logical cores and was configured in such a way that the feasibility pump heuristic was used exclusively. CP-SAT was executed on a server using 64 logical cores. In the low-λ regime, we used the heuristic bounds $\widetilde{\nu}$ for the u-variables as indicated in Remark 20, although the effect seems to be limited. The results suggest that Gurobi is more efficient at higher bit indices, whereas CP-SAT performs better for low bit indices. However, the comparison should be interpreted cautiously, because different hardware was used.

Remark 24. In our experiments, we were unable to solve ILP 2 except for very small error probabilities p_{err}. We also implemented some custom primal heuristics based on the observation in Remark 23. However, even with the correct solution provided as a hint, the solvers were not capable of proving optimality of the solution in a reasonable amount of time. Therefore, we suspect that general-purpose ILP solvers are not adequate for solving Problem 1 with noisy leakage, unless major improvements can be found. In Section 6, we develop a tailored attack based on nonlinear programming that performs very well even in the presence of substantial noise.

6 Nonlinear programming

Fix a key $x^* \in \mathcal{X}$ and let $(c^{(i)}, z^{(i)}, \widetilde{y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}}$ be leaky signatures generated by independent runs of Algorithm 4.

Using Lemma 43 (b) and (d), the most likely keys are the solutions to the optimization problem

$$\mathcal{X}_{\mathsf{ml}} = \operatorname*{arg\,min}_{x \in \mathcal{X}} \textstyle\sum_{i=1}^{n_{\mathsf{sig}}} \psi\big(t^{(i)}(x)\big)\,, \tag{NLP}$$

where $\psi(t) = 1_{[-2^\lambda, 0)}(t \bmod^\mp 2^{\lambda+1})$ and $t^{(i)}(x) = z^{(i)} - \widetilde{y}_\lambda{}^{(i)} 2^\lambda - \langle c^{(i)}, x\rangle + \frac{1}{2}$. The shift by $\frac{1}{2}$ was chosen to achieve symmetry and maximize the distance between the images $t^{(i)}(\mathcal{X}) \subset \frac{1}{2} + \mathbb{Z}$ and the points of discontinuity of ψ, all of which are integral.

In this way, when we approximate ψ by a continuous function ψ_{approx} in Section 6.1, the approximation of the composed functions $\psi_{\mathsf{approx}}\big(t^{(i)}(x)\big) \approx \psi\big(t^{(i)}(x)\big)$ at points $x \in \mathcal{X}$ is improved.

6.1 Continuous optimization

Instead of solving (NLP) directly, we may consider the unconstrained continuous problem

$$\operatorname*{arg\,min}_{x \in \mathbb{R}^{256}} \textstyle\sum_{i=1}^{n_{\mathsf{sig}}} \psi_{\mathsf{approx}}\big(t^{(i)}(x)\big) + \psi_{\mathsf{penalty}}(x)\,, \tag{NLP$_{\mathsf{approx}}$}$$

where $\psi_{\mathsf{approx}} \colon \mathbb{R} \to \mathbb{R}$ is an approximation of the step function ψ and $\psi_{\mathsf{penalty}} \colon \mathbb{R}^{256} \to \mathbb{R}$ is a function designed to attain minimal values near $\mathcal{X}$. We then hope that solutions x to the transformed problem are close to $\mathcal{X}_{\mathsf{ml}}$, and ideally, that the rounded values $\lfloor x \rceil$ lie in $\mathcal{X}_{\mathsf{ml}}$.

Differentiable approximation

Since the function $\psi(t) = 1_{[-2^\lambda, 0)}(t \bmod^\mp 2^{\lambda+1})$ is $2^{\lambda+1}$-periodic and, except at its points of discontinuity, symmetric about the axis $t = 2^{\lambda-1}$, it is essentially determined by its restriction to $[-2^{\lambda-1}, 2^{\lambda-1}]$, where it coincides with the indicator function $1_{[-2^{\lambda-1}, 0)}$.

It is therefore natural to select for the restriction $\psi_{\mathsf{approx}}|_{[-2^{\lambda-1}, 2^{\lambda-1}]}$ a suitable continuous approximation of $1_{[-2^{\lambda-1}, 0)}$, and to extend it to a $2^{\lambda+1}$-periodic function $\psi_{\mathsf{approx}} \colon \mathbb{R} \to \mathbb{R}$ that is symmetric about the axis $t = 2^{\lambda-1}$. To apply gradient-based algorithms, we require the function ψ_{approx} to be not only continuous but also differentiable almost everywhere.

Concretely, once the restriction $\psi_{\mathsf{approx}}|_{[-2^{\lambda-1}, 2^{\lambda-1}]}$ has been chosen satisfying these properties, its extension $\psi_{\mathsf{approx}} \colon \mathbb{R} \to \mathbb{R}$ is computed as

$$\psi_{\mathsf{approx}}(t) = \begin{cases} \psi_{\mathsf{approx}}|_{[-2^{\lambda-1}, 2^{\lambda-1}]}(t \bmod^\mp 2^{\lambda+1}) & \text{if } t \bmod^\mp 2^{\lambda+1} \in [-2^{\lambda-1}, 2^{\lambda-1})\,, \\ \psi_{\mathsf{approx}}|_{[-2^{\lambda-1}, 2^{\lambda-1}]}((2^\lambda - t) \bmod^\mp 2^{\lambda+1}) & \text{otherwise}\,. \end{cases}$$

Remark 25. In practice, many choices for $f = \psi_{\mathsf{approx}}|_{[-2^{\lambda-1}, 2^{\lambda-1}]}$ with a moderate slope near $t = 0$ lead to satisfactory results; for example, the following functions with slope $\approx -\frac{1}{2}$ at $t = 0$ work well:

(a) $f(t) = \Phi(-kt)$ with $k \approx \sqrt{\pi/2}$

(b) $f(t) = 1/(1 + \exp(kt))$ with $k \approx 2$

(c) $f(t) = \frac{1}{2} + \frac{1}{\pi}\arctan(-kt)$ with $k \approx \pi/2$

(d) the squared version of any of (a) – (c)

The approximations (a) – (c) are the complementary cumulative distribution functions of the normal distribution with variance $1/k^2$, of the logistic distribution with parameter $1/k$, and of the Cauchy distribution with parameter $1/k$, respectively, all centered around 0; see [Heaviside] for further details on approximations of the Heaviside step function. In our experiments, we primarily used the squared version of approximation (a), as it yielded slightly better results than using (a) directly. For a visualization of the approximations, see Figure 7.

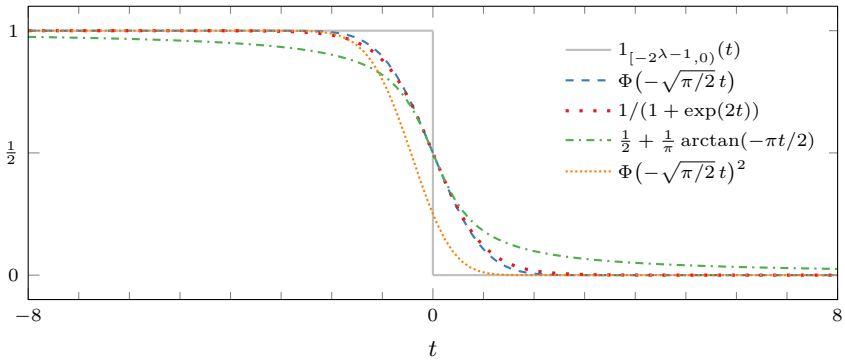


Figure 7: Examples for continuous approximations ψ_{approx} of ψ on $[-2^{\lambda-1}, 2^{\lambda-1}]$.

Remark 26. Figure 8 presents exemplary coordinate slices of the objective function for the optimization problem (NLP), contrasting the formulation in terms of the exact ψ with the corresponding version based on the approximation ψ_{approx}. We observe that, when all but one coordinate are fixed to the true key (plots on the left), even a relatively small number n_{sig} of signatures yields a pronounced minimum at the value of the true key. Interestingly, when $\lambda \geq 4$ for ML-DSA-44 and ML-DSA-87, or when $\lambda \geq 5$ for ML-DSA-65, once n_{sig} becomes sufficiently large, a clear minimum still emerges at the true key even when the remaining coordinates are chosen at random (plots on the right).

Soft constraints

As the variable x varies continuously over all of $\mathbb{R}^{256}$ in ($\mathrm{NLP}_{\mathsf{approx}}$), the term $\psi_{\mathsf{penalty}}(x)$ in the objective function is intended to increase the likelihood that solutions are close to $\mathcal{X}$, and thereby, hopefully, also closer to $\mathcal{X}_{\mathsf{ml}}$ than they would be without this term.

Remark 27. In practice, when n_{sig} is sufficiently large, even setting $\psi_{\mathsf{penalty}} = 0$ turns out to be a reasonable choice. For smaller values of n_{sig}, using an appropriate ψ_{penalty} can significantly improve the chance of successful key recovery. In our experiments, we took

$$\psi_{\mathsf{penalty}}(x) = w_{\mathsf{int}}\,\psi_{\mathsf{int}}(x) + w_{\mathsf{box}}\,\psi_{\mathsf{box}}(x)\,,$$

where $\psi_{\mathsf{int}}(x) = \sum_{j=1}^{256} \sin(\pi x_j)^2$ and $\psi_{\mathsf{box}}(x) = \sum_{j=1}^{256}(\Phi(k(-\eta_+ - x_j)) + \Phi(-k(\eta_+ - x_j)))$. The functions ψ_{int} and ψ_{box} serve to penalize violations of integrality and box constraints, respectively. Typical parameter settings included $w_{\mathsf{int}} \approx \frac{1}{5}$, $w_{\mathsf{box}} \approx 5$, $\eta_+ \approx \eta + \frac{1}{4}$, $k \approx 40$; see Figure 9 for an illustration.

6.2 Combinatorial optimization

Next, we try to solve (NLP) directly. Let G be a directed graph with vertex set $\mathcal{X}$ and vertex weights

$$w(x) = \#\big\{i \in [n_{\mathsf{sig}}] \mid \widetilde{y}_\lambda{}^{(i)} \neq \mathrm{bit}_\lambda(z^{(i)} - \langle c^{(i)}, x\rangle)\big\}$$

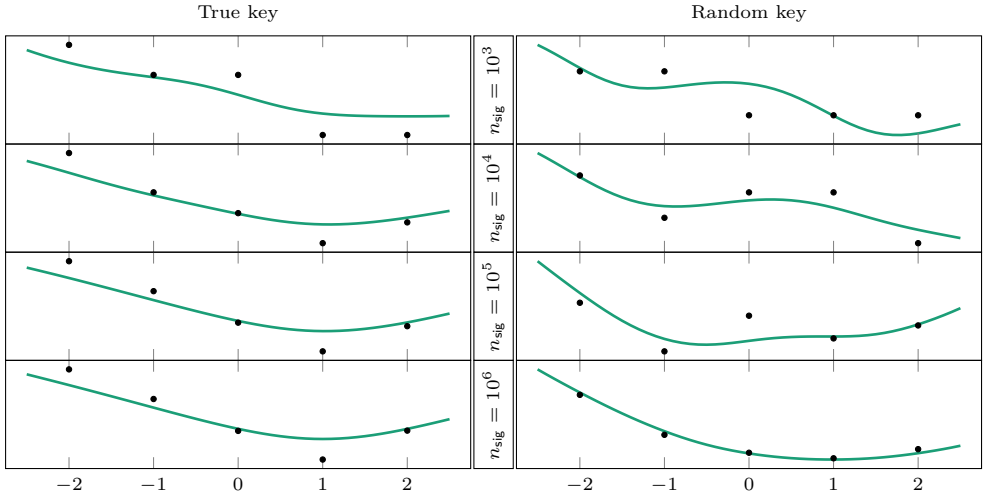


Figure 8: Coordinate slices $x_j \mapsto \sum_{i=1}^{n_{\mathsf{sig}}} \psi\big(t^{(i)}(x)\big)$ (dots) and $x_j \mapsto \sum_{i=1}^{n_{\mathsf{sig}}} \psi_{\mathsf{approx}}\big(t^{(i)}(x)\big)$ (curves, scaled) with the other coordinates fixed either to the coordinates $(x^*_{j'})_{j' \neq j}$ of the true key (plots on the left) or to a random vector with entries in $[-\eta, \eta] \cap \mathbb{Z}$ (plots on the right); example for the parameter set ML-DSA-44 with $\lambda = 6$, $p_{\mathsf{err}} = 0$, and $x^*_j = 1$.

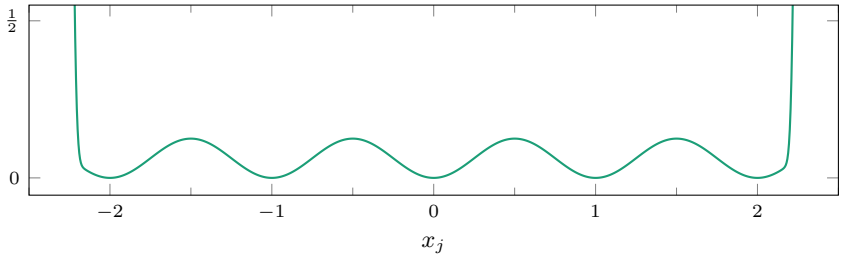


Figure 9: Coordinate slices $\psi_{\mathsf{penalty}}(0, \ldots, x_j, \ldots, 0)$ in the situation $\eta = 2$ with parameter values $w_{\mathsf{int}} = \frac{1}{5}$, $w_{\mathsf{box}} = 5$, $\eta_+ = \eta + \frac{1}{4}$, $k = 40$.

such that the minimal-weight vertices form the set $\mathcal{X}_{\mathsf{ml}}$ of most likely keys; see Definition 39. We can start a graph search at some vertex in $\mathcal{X}$ with the goal of reaching a vertex in $\mathcal{X}_{\mathsf{ml}}$. For simplicity, we define the set of edges in G as

$$\big\{(x, x') \in \mathcal{X}^2 \mid \|x - x'\|_1 = 1, w(x') \leq w(x)\big\} \ ,$$

and traverse the graph using a greedy best-first strategy: at each step, we process the lowest-weight reached vertex that remains unprocessed, expanding the search to its neighbors.

Remark 28. In practice, when n_{sig} is sufficiently large such that, in particular, $\mathcal{X}_{\mathsf{ml}} = \{x^*\}$ with high probability, the graph search typically reaches x^*. The search can be significantly accelerated by starting at a vertex near a solution to the continuous problem ($\mathrm{NLP}_{\mathsf{approx}}$).

Remark 29. For all $x, x' \in \mathcal{X}$ satisfying $\|x' - x^*\|_1 < \|x - x^*\|_1$ and, moreover, $\|x - x'\|_1 = 1$, so that $x_j \neq x'_j$ for a unique index j, we have in the high-λ regime

$$\mathbb{P}\big(\mathrm{bit}_\lambda(Z - \langle C, x\rangle) \neq \mathrm{bit}_\lambda(Z - \langle C, x^*\rangle) = \mathrm{bit}_\lambda(Z - \langle C, x'\rangle)\big) = \tfrac{p_{\mathsf{inf}}}{2\beta}\tfrac{\tau}{256} \cdot \mathbb{P}(T < t) \, ,$$
$$\mathbb{P}\big(\mathrm{bit}_\lambda(Z - \langle C, x\rangle) = \mathrm{bit}_\lambda(Z - \langle C, x^*\rangle) \neq \mathrm{bit}_\lambda(Z - \langle C, x'\rangle)\big) = \tfrac{p_{\mathsf{inf}}}{2\beta}\tfrac{\tau}{256} \cdot \mathbb{P}(T \geq t) \, ,$$

where T denotes the random variable $\big(\sum_{i \neq j} C_i(x^*_i - x_i) \mid C_j \neq 0\big)$ and $t = |x^*_j - x_j| > 0$. We refer to Section A.1 for the precise definition of the random variables C and Z. Note that the first probability is always at least as large as the second and, for most pairs x

and x', it should in fact be strictly larger. This observation makes it plausible that, for sufficiently large n_{sig}, the graph search described above should indeed terminate at the true key x^* after not much more than $\|x_0 - x^*\|_1$ steps, when started at the vertex x_0.

6.3 Experiments

In each experiment, we assume that the bit index λ and the error probability p_{err} are fixed, and informative leaky signatures $(c^{(i)}, z^{(i)}, \widetilde{y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{inf}}}$ are given, having been generated by Algorithm 4 with input x^*. To emphasize that only informative signatures are considered, we denote their count by n_{inf} rather than n_{sig}. By Lemma 53 (b), the expected number of leaky signatures required to obtain n_{inf} informative signatures is $\mathbb{E}(N_{\mathsf{sig}}) = n_{\mathsf{inf}}/p_{\mathsf{inf}}$, where p_{inf} is given by Lemma 49 (b). The goal is to recover the key x^* using only the signatures and the known values of λ. We approached each experiment as follows:

(i) Solve the optimization problem ($\mathrm{NLP}_{\mathsf{approx}}$) repeatedly, varying the functions ψ_{approx} and ψ_{penalty}, and the initial value of x in each repetition.

(ii) Project the solutions found in (i) onto $\mathcal{X}$ by rounding, and let $\mathcal{X}_0$ denote the resulting set of key approximations. Select $x_0 \in \arg\min_{x \in \mathcal{X}_0} w(x)$.

(iii) Start the graph search, as described in Section 6.2, from x_0. After running the search for a limited time, return the vertex $\widehat{x}$ with the lowest weight encountered. An experiment is regarded as successful if $\widehat{x}$ matches the true key x^*.

We implemented this NLP attack using the Julia programming language [Julia]. To solve the continuous optimization problem, we employed the quasi-Newton method `LBFGS` from the `Optim.jl` package [Optim.jl], to which we passed all gradient evaluations explicitly within our implementation. We additionally tested the first-order methods `AdaMax`, `ConjugateGradient`, and `GradientDescent`, the quasi-Newton method `BFGS`, and the second-order method `Newton` provided by the same package. While the alternative methods usually yielded similarly reliable approximations, `LBFGS` tended to run more efficiently.

Figure 10 and Figure 11 present our experimental results across the various parameter sets, for exact and noisy leakage respectively. Reliable key recovery using this attack was achieved when $\lambda \geq 4$ for ML-DSA-44 and ML-DSA-87, and when $\lambda \geq 5$ for ML-DSA-65. For exact leakage, key recovery still worked for $\lambda = 3$ and $\lambda = 4$, respectively, but required a large number of signatures.

We performed this intricate procedure, consisting of steps (i) – (iii), solely to minimize the number of required signatures. With enough signatures available, a single run of either the continuous or combinatorial method typically suffices to recover the key. For noisy leakage with high error probabilities $p_{\mathsf{err}} > 0.45$, to accelerate the computations, we did not minimize the number of signatures but instead tested whether all experiments succeeded using the heuristic upper bound $n_{\mathsf{inf,upper}}/\big(1 - 2\sqrt{(1-p_{\mathsf{err}})p_{\mathsf{err}}}\big)$; see Remark 64, Remark 66, and Figure 1 for the values of $n_{\mathsf{inf,upper}}$. This test was successful except for ML-DSA-87 with $\lambda = 4$ (where we adjusted the number of signatures by factors of 1.8 and 2, respectively, for $0.45 < p_{\mathsf{err}} < 0.49$ and $p_{\mathsf{err}} = 0.49$), and for a single outlier at $p_{\mathsf{err}} = 0.48$ for ML-DSA-65 with $\lambda = 8$ (where we applied an adjustment factor of 1.05).

Remark 30. In the case of exact leakage, a single run of the continuous and combinatorial method typically finishes in just a few seconds for sensible values of n_{inf}. For example, key recovery for ML-DSA-44 with $(\lambda, n_{\mathsf{inf}}) \in \{(4, 1\,500), (5, 3\,000), (6, 6\,000), (7, 9\,000), (8, 12\,000)\}$ typically took a few seconds and never exceeded one minute on a laptop (single-threaded execution). When n_{inf} is taken closer to the minimum value that still permits key recovery, the runtime of the combinatorial method increases significantly, while the continuous method may require more careful parameter tuning to succeed at all.

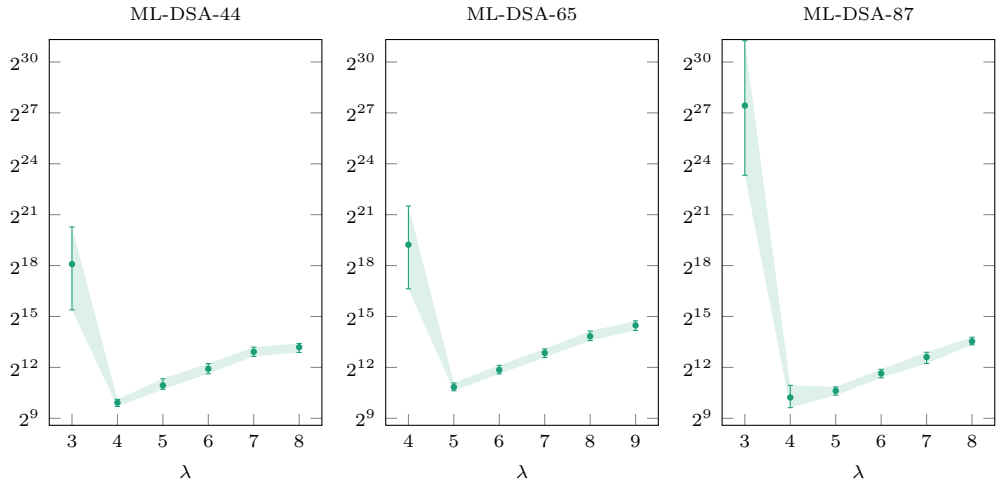


Figure 10: Number of informative signatures with exact leakage required for key recovery using the NLP attack: minimum, maximum, and average value across 100 experiments.

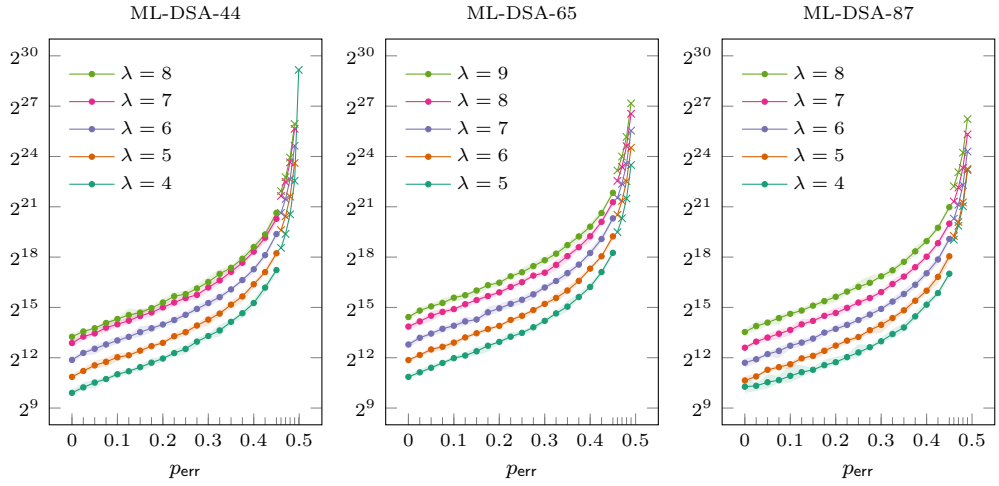


Figure 11: Number of informative signatures with noisy leakage required for key recovery using the NLP attack: minimum, maximum, and average value across 10 experiments per data point (dots), and value for which all out of 10 experiments succeeded (crosses).

Key recovery still worked for ML-DSA-44 and ML-DSA-87 with $\lambda = 3$ and for ML-DSA-65 with $\lambda = 4$, but only given a relatively large number of signatures. This raises the question for which λ in the low-λ regime practical attacks are possible. But note that there are theoretical limits, e.g., the key cannot be fully recovered for $\lambda = 0$, where its entries can only be determined modulo 2.

Remark 31. In the case of noisy leakage, by Remark 66, the number of informative signatures required for key recovery increases roughly proportional to $1/\left(1 - 2\sqrt{(1 - p_{\mathsf{err}})p_{\mathsf{err}}}\right)$, and we expect the runtime to grow accordingly. In experiments running the continuous method followed by the combinatorial method, for ML-DSA-44 with error probability $p_{\mathsf{err}} = 0.45$ and parameters $(\lambda, n_{\mathsf{inf}}) \in \{(4, 200\,000), (5, 400\,000), (6, 800\,000), (7, 1\,600\,000), (8, 3\,200\,000)\}$, recovering the key usually completed in a few minutes, not exceeding ten minutes.

Remark 32. With sufficient time and memory available, key recovery remains feasible even for relatively high values of p_{err}. For example, $p_{\mathsf{err}} = 0.49$ is still achievable for ML-DSA-44 with $\lambda = 4$ and $n_{\mathsf{inf}} = 6\,000\,000$, taking less than ten minutes on a laptop (32 GB RAM).

Running the computation on a server (single-threaded, 512 GB RAM), $p_{\mathsf{err}} = 0.49$ is achievable also for ML-DSA-44 and ML-DSA-87 with $\lambda = 8$ and $n_{\mathsf{inf}} = 100\,000\,000$, as well as for ML-DSA-65 with $\lambda = 9$ and $n_{\mathsf{inf}} = 150\,000\,000$, each taking less than an hour. The most favorable case is ML-DSA-44 with $\lambda = 4$. Here, the number of required signatures is especially small, and storage is more efficient as well, since for leaky signatures $(c, z, \widetilde{y}_\lambda)$ the value $z \bmod^{\mp} 2^{\lambda+1}$ fits into one byte. In this setting, even for $p_{\mathsf{err}} = 0.499$, key recovery was still practicable with $n_{\mathsf{inf}} = 600\,000\,000$ in less than six hours, dropping to under ten minutes when parallelized across 64 threads.

Note that these values are significantly better than the maximum value $p_{\mathsf{err}} = 0.43$ reported in [Dam+26, Section 8] and $p_{\mathsf{err}} = 0.45$ reported in [Sch+26, Table 4].

The source code of our NLP implementation is available at

https://github.com/heurekos/LeakyMLDSA.jl .

Remark 33. It is also possible to use belief propagation (BP) to exploit single-bit leakage. This was proposed by the anonymous reviewers and stated as future research direction in [Sch+26]. In this context, [Her+25] introduced an algorithm based on BP that applies to leakage models where the conditional distribution of the scalar product $S = \langle C, X^* \rangle$, given the signature and leakage, is available (see Lemma 36 for the definition of S).

We used the implementation accompanying [Her+25] for a brief comparison of this BP approach and our NLP attack in terms of the number of signatures required for key recovery. We carried out a small number of experiments for each ML-DSA parameter set across representative values of λ and $p_{\mathsf{err}} \in \{0, 0.2\}$. The two attacks required a similar number of signatures in almost all cases. The average relative difference was typically within $\pm 5\%$ and always within $\pm 10\%$, except for ML-DSA-87 with $\lambda = 4$. For this outlier, the NLP attack required significantly more signatures than predicted by the heuristics shown in Figure 1 and Figure 2, while the BP attack remained close to the prediction. For details, see https://heurekos.github.io/LeakyMLDSA.jl/v0.0.2/bp-comparison/.

Because of high memory requirements, we did not run BP experiments for ML-DSA-44 and ML-DSA-87 with $\lambda = 3$ or for ML-DSA-65 with $\lambda = 4$ using many signatures, such as in Remark 30. Whether the BP attack works in this regime remains an open question.

7 Conclusion

We analyzed a leakage attack introduced in [Liu+21] and improved in [Dam+25a; Dam+26]. Through new attack methods based on integer linear programming and nonlinear programming, we substantially reduced the number of leaky signatures necessary for key recovery in practice. We supported these results with a thorough theoretical analysis, indicating that the number of signatures required for our attacks is nearly optimal.

Acknowledgments. We thank Tobias Hemmert for interesting discussions, Friederike Laus for proofreading an earlier version of this paper, and the anonymous reviewers for valuable comments that improved the presentation of the paper.

A Probabilistic and information-theoretic considerations

In Section A.1, we introduce the stochastic model used for our theoretical analysis. Section A.2 defines the sets of candidate keys and most likely keys and Section A.3 examines information-theoretic properties. We then provide estimates for the expected number of candidate keys and most likely keys in Section A.4 and Section A.5, respectively, which yields estimates for the number of signatures required for key recovery.

A.1 Stochastic model

We define a stochastic model for the random elements sampled in Problem 1, particularly during executions of Algorithm 4. To this end, we introduce the following notation.

We fix a bit index $\lambda \geq 0$ and an error probability $p_{\mathsf{err}} \in [0, \frac{1}{2})$. Let $x^* \in \mathcal{X}$ be a key. For $i \geq 1$, we denote by $(c^{(i)}, z^{(i)}, \widetilde{y}_\lambda{}^{(i)})$ the output of the i-th run of Algorithm 4 with input x^*. We also set $y^{(i)} := z^{(i)} - \langle c^{(i)}, x^* \rangle$, $y_\lambda{}^{(i)} := \mathrm{bit}_\lambda(y^{(i)})$, and $e^{(i)} := y_\lambda{}^{(i)} \oplus \widetilde{y}_\lambda{}^{(i)}$ for all $i \geq 1$. The variables sampled in the j-th loop iteration of the i-th run of Algorithm 4 with input x^* are denoted by $y^{(i)[j]}$ and $c^{(i)[j]}$, and we set $z^{(i)[j]} := \langle c^{(i)[j]}, x^* \rangle + y^{(i)[j]}$ for all $i, j \geq 1$. We view these as realizations of the random variables defined below.

Definition 34 (Stochastic model)**.** Let $\lambda \geq 0$ and $p_{\mathsf{err}} \in [0, \frac{1}{2})$.

(a) Let $\{X^*\} \cup \{Y^{(i)[j]}, C^{(i)[j]}\}_{i,j \geq 1} \cup \{E^{(i)}\}_{i \geq 1}$ be an independent family of random variables such that

- X^* is uniformly distributed on $\mathcal{X} := ([-\eta, \eta] \cap \mathbb{Z})^{256}$,
- $Y^{(i)[j]}$ is uniformly distributed on $\mathcal{Y}' := (-\gamma, \gamma] \cap \mathbb{Z}$ for all $i, j \geq 1$,
- $C^{(i)[j]}$ is uniformly distributed on $\mathcal{C} := \{c \in \mathbb{Z}^{256} \mid \|c\|_\infty = 1 \text{ and } \|c\|_1 = \tau\}$ for all $i, j \geq 1$,
- $E^{(i)}$ is Bernoulli-distributed with $\mathbb{P}(E^{(i)} = 1) = p_{\mathsf{err}}$ for all $i \geq 1$.

For $i, j \geq 1$, define the random variables $Z^{(i)[j]} := \langle C^{(i)[j]}, X^* \rangle + Y^{(i)[j]}$ taking values in $\mathcal{Z}' := (-\gamma - \beta, \gamma + \beta] \cap \mathbb{Z}$.

(b) Let $\alpha(z) := 1_{(-\gamma+\beta, \gamma-\beta)}(z)$ for $z \in \mathbb{R}$ and define the random variables

$$J(i) := \inf\{j \geq 1 \mid \alpha(Z^{(i)[j]}) = 1\}, \qquad i \geq 1,$$

i.e. $J(i)$ is the random index of the final loop iteration in the i-th run of Algorithm 4.

(c) For $i \geq 1$, define the random variables

- $C^{(i)} := C^{(i)[J(i)]}$ taking values in $\mathcal{C}$,
- $Z^{(i)} := Z^{(i)[J(i)]}$ taking values in $\mathcal{Z} := (-\gamma + \beta, \gamma - \beta) \cap \mathbb{Z}$,
- $Y^{(i)} := Y^{(i)[J(i)]}$ taking values in $\mathcal{Y} := (-\gamma, \gamma) \cap \mathbb{Z}$,
- $Y_\lambda{}^{(i)} := \mathrm{bit}_\lambda(Y^{(i)})$ taking values in $\{0, 1\}$,
- $\widetilde{Y}_\lambda{}^{(i)} := Y_\lambda{}^{(i)} \oplus E^{(i)}$ taking values in $\{0, 1\}$.

If just a single run of Algorithm 4 is under consideration, we omit the index "(i)" indicating the i-th run. We also write Y' and C' for independent copies of the random variables $Y^{(i)[j]}$ and $C^{(i)[j]}$ (independent of X^*) and we define $Z' := \langle C', X^* \rangle + Y'$.

We start with some immediate consequences of these definitions in the following lemma. Assertion (a) is [Dam+25a, Lemma 2 and Eq. (1)] adapted to the support $\mathcal{Y}'$ of Y'.

Lemma 35. *The random variables in Definition 34 satisfy the following properties.*

(a) The random variables $\{Z^{(i)[j]}\}_{i,j\geq 1}$ *have the probability mass function*

$$\begin{aligned}\mathbb{P}\big(Z'=z\big) &= \tfrac{1}{2\gamma}\cdot\mathbb{P}\big(z-\gamma\leq\langle C',X^*\rangle\leq z+\gamma-1\big)\\ &= \tfrac{1}{2\gamma}\cdot\begin{cases}\mathbb{P}\big(\langle C',X^*\rangle\leq z+\gamma-1\big) & \text{if } -\gamma-\beta<z\leq-\gamma+\beta\,,\\ 1 & \text{if } -\gamma+\beta<z\leq\gamma-\beta\,,\\ \mathbb{P}\big(\langle C',X^*\rangle\geq z-\gamma\big) & \text{if } \gamma-\beta<z\leq\gamma+\beta\,.\end{cases}\end{aligned}$$

(b) The family $\{X^*\}\cup\{C^{(i)[j]},\alpha(Z^{(i)[j]})\}_{i,j\geq 1}$ *of random variables is independent.*

(c) The random variables $\{J(i)\}_{i\geq 1}$ *are geometrically distributed with support* $\mathbb{N}_{\geq 1}$ *and success probability* $\mathbb{P}\big(\alpha(Z')=1\big)=\frac{2(\gamma-\beta)-1}{2\gamma}$.

(d) The family $\{X^*\}\cup\{C^{(i)},Z^{(i)},E^{(i)},J(i)\}_{i\geq 1}$ *of random variables is independent.*

(e) The random variables $\{C^{(i)}\}_{i\geq 1}$ *are uniformly distributed on* $\mathcal{C}$.

(f) The random variables $\{Z^{(i)}\}_{i\geq 1}$ *are uniformly distributed on* $\mathcal{Z}$.

Proof. Since X^*, C', and Y' are independent and $Z'=\langle C',X^*\rangle+Y'$, we obtain

$$\begin{aligned}\mathbb{P}\big(X^*=x,C'=c,Z'=z\big) &= \textstyle\sum_{y\in\mathcal{Y}'}\mathbb{P}\big(X^*=x,C'=c,\langle C',X^*\rangle=z-y\big)\cdot\mathbb{P}\big(Y'=y\big)\\ &= \tfrac{1}{2\gamma}\cdot\mathbb{P}\big(X^*=x,C'=c,z-\gamma\leq\langle C',X^*\rangle\leq z+\gamma-1\big)\,.\end{aligned}$$

Summing this equation over all $x\in\mathcal{X}$ and $c\in\mathcal{C}$ gives the first identity in (a). The fact that $\langle C',X^*\rangle$ takes values in $[-\beta,\beta]$ yields the second identity in (a) and, together with the independence of X^* and C', also shows that

$$\mathbb{P}\big(X^*=x,C'=c,Z'=z,\alpha(Z')=1\big)=\mathbb{P}(X^*=x)\cdot\mathbb{P}(C'=c)\cdot\tfrac{\alpha(z)}{2\gamma}\,.$$

Summing over all $z\in\mathcal{Z}$ yields

$$\mathbb{P}\big(X^*=x,C'=c,\alpha(Z')=1\big)=\mathbb{P}(X^*=x)\cdot\mathbb{P}(C'=c)\cdot\tfrac{2(\gamma-\beta)-1}{2\gamma}\,,$$

which implies the independence of $\{X^*,C',\alpha(Z')\}$. The independence of any finite subset of $\{X^*\}\cup\{C^{(i)[j]},\alpha(Z^{(i)[j]})\}_{i,j\geq 1}$ can be shown analogously, thereby proving (b). Summing the probabilities $\mathbb{P}\big(X^*=x,C'=c,\alpha(Z')=1\big)$ over all $x\in\mathcal{X}$ and $c\in\mathcal{C}$, we obtain $\mathbb{P}\big(\alpha(Z')=1\big)=\frac{2(\gamma-\beta)-1}{2\gamma}$, establishing (c). The calculation of the joint distribution

$$\begin{aligned}&\mathbb{P}\big(X^*=x,C=c,Z=z,E=e,J=j\big)\\ &\quad=\mathbb{P}\big(X^*=x,C^{[j]}=c,Z^{[j]}=z,E=e,J=j\big)\\ &\quad=\mathbb{P}\big(X^*=x,C^{[j]}=c,Z^{[j]}=z,E=e,\alpha(Z^{[1]})=\cdots=\alpha(Z^{[j-1]})=0,\alpha(Z^{[j]})=1\big)\\ &\quad=\mathbb{P}\big(X^*=x,C'=c,Z'=z,\alpha(Z')=1\big)\cdot\mathbb{P}\big(E=e\big)\cdot\mathbb{P}\big(\alpha(Z')=0\big)^{j-1}\\ &\quad=\mathbb{P}\big(X^*=x\big)\cdot\mathbb{P}\big(C'=c\big)\cdot\tfrac{\alpha(z)}{2\gamma}\cdot\mathbb{P}\big(E=e\big)\cdot\mathbb{P}\big(\alpha(Z')=0\big)^{j-1}\\ &\quad=\mathbb{P}\big(X^*=x\big)\cdot\mathbb{P}\big(C'=c\big)\cdot\tfrac{\alpha(z)}{2(\gamma-\beta)-1}\cdot\mathbb{P}\big(E=e\big)\cdot\mathbb{P}\big(\alpha(Z')=0\big)^{j-1}\cdot\mathbb{P}\big(\alpha(Z')=1\big)\\ &\quad=\mathbb{P}\big(X^*=x\big)\cdot\mathbb{P}\big(C'=c\big)\cdot\tfrac{\alpha(z)}{2(\gamma-\beta)-1}\cdot\mathbb{P}\big(E=e\big)\cdot\mathbb{P}\big(J=j\big)\end{aligned}$$

implies the independence of $\{X^*,C,Z,E,J\}$ and claims (e) – (f). The independence of any finite subset of $\{X^*\}\cup\{C^{(i)},Z^{(i)},E^{(i)},J(i)\}_{i\geq 1}$ can be shown analogously, finishing the proof of (d). □

The set $\mathcal{C}$ consists of all vectors in $\{-1,0,1\}^{256}$ with exactly τ non-zero components, hence $\#\mathcal{C}=2^\tau\cdot\binom{256}{\tau}$. The following simple lemma shows that the non-zero components and the positions of those components are independent.

Lemma 36. *Let $J_1 < \cdots < J_\tau$ be the elements of $\mathfrak{J} := \{j \in [256] \mid C_j \neq 0\}$, the random τ-subset of indices where C does not vanish. Then $\mathfrak{J}$ is uniformly distributed on $\binom{[256]}{\tau}$, $C_{J_1}, \ldots, C_{J_\tau}$ are uniformly distributed on $\{-1, 1\}$, and $\mathfrak{J}, C_{J_1}, \ldots, C_{J_\tau}$ are independent.*

Proof. This is due to the bijection $\mathcal{C} \to \binom{[256]}{\tau} \times \{-1, 1\}^\tau$, $c \mapsto \big(\{j_1, \ldots, j_\tau\}, (c_{j_1}, \ldots, c_{j_\tau})\big)$, where $j_1 < \cdots < j_\tau$ are the elements of $\{j \in [256] \mid c_j \neq 0\}$. □

The next lemma states some properties of the random variable $S := \langle C, X^* \rangle$. Assertion (b) shows how to efficiently compute its probability mass function.

Lemma 37. *Define the random variable $S := \langle C, X^* \rangle$ taking values in $\mathcal{S} := [-\beta, \beta] \cap \mathbb{Z}$.*

(a) The random variable S can be written as $S = U_1 + \cdots + U_\tau$, where $U_1, \ldots, U_\tau$ are uniformly distributed random variables on $[-\eta, \eta] \cap \mathbb{Z}$ such that $C, U_1, \ldots, U_\tau$ are independent. In particular, C and S are independent.

(b) We have $\mathbb{P}(S = s) = \frac{\xi(s)}{(2\eta+1)^\tau}$ for all $s \in \mathcal{S}$, where $\xi(s)$ is the coefficient of the monomial t^s in the Laurent polynomial $\big(\sum_{u=-\eta}^{\eta} t^u\big)^\tau \in \mathbb{Z}[t, t^{-1}]$.

Proof. By Lemma 36, we have $S = C_{J_1} X^*_{J_1} + \cdots + C_{J_\tau} X^*_{J_\tau}$. Define $U_k := C_{J_k} X^*_{J_k}$ for $k \in [\tau]$. Let $c \in \mathcal{C}$ and $u_1, \ldots, u_\tau \in [-\eta, \eta] \cap \mathbb{Z}$. As before, denote by $j_1 < \cdots < j_\tau$ the elements of the set $\{j \in [256] \mid c_j \neq 0\}$. Using the independence of C and X^*, we obtain

$$\begin{aligned}\mathbb{P}\big(C = c, U_1 = u_1, \ldots, U_\tau = u_\tau\big) &= \mathbb{P}\big(C = c, X^*_{j_1} = u_1/c_{j_1}, \ldots, X^*_{j_\tau} = u_\tau/c_{j_\tau}\big) \\ &= \mathbb{P}\big(C = c\big) \cdot \textstyle\prod_{k=1}^{\tau} \mathbb{P}\big(X_{j_k} = u_k/c_{j_k}\big)\,,\end{aligned}$$

which is independent of $c, u_1, \ldots, u_\tau$. This concludes the proof of assertion (a) and claim (b) is a straightforward consequence of (a). □

The following lemma collects several results concerning the joint distributions of key and leaky signatures.

Lemma 38. *Let $n_{\mathsf{sig}} \geq 0$.*

(a) Denoting $T = (C^{(i)}, Z^{(i)}, \widetilde{Y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}}$ and $t = (c^{(i)}, z^{(i)}, \widetilde{y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}}$, we have

$$\mathbb{P}\big(X^* = x, T = t\big) = \frac{\prod_{i=1}^{n_{\mathsf{sig}}} \mathbb{P}\big(E = \widetilde{y}_\lambda{}^{(i)} \oplus \mathrm{bit}_\lambda(z^{(i)} - \langle c^{(i)}, x \rangle)\big)}{\#\mathcal{X} \cdot (\#\mathcal{C} \cdot \#\mathcal{Z})^{n_{\mathsf{sig}}}}\,.$$

In particular, the family $\{(C^{(i)}, Z^{(i)}, \widetilde{Y}_\lambda{}^{(i)})\}_{i \geq 1}$ of random tuples is conditionally independent given the random key X^*.

(b) We have $\mathbb{P}\big(\widetilde{Y}_\lambda = b \mid C = c, Z = z\big) = \mathbb{P}\big(\widetilde{Y}_\lambda = b \mid Z = z\big)$, where

$$\mathbb{P}\big(\widetilde{Y}_\lambda = b \mid Z = z\big) = (1 - p_{\mathsf{err}})\, \mathbb{P}\big(b = \mathrm{bit}_\lambda(z - S)\big) + p_{\mathsf{err}}\, \mathbb{P}\big(b \neq \mathrm{bit}_\lambda(z - S)\big)\,.$$

In particular, the random variable C and the random tuple $\big(Z, \widetilde{Y}_\lambda\big)$ are independent.

Proof. As $\widetilde{Y}_\lambda{}^{(i)} = E^{(i)} \oplus \mathrm{bit}_\lambda(Z^{(i)} - \langle C^{(i)}, X^* \rangle)$ for all $i \in [n_{\mathsf{sig}}]$, we have

$$\begin{aligned}&\mathbb{P}\big(X^* = x, (C^{(i)}, Z^{(i)}, \widetilde{Y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}} = (c^{(i)}, z^{(i)}, \widetilde{y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}}\big) \\ &\quad = \mathbb{P}\big(X^* = x,\ \textstyle\bigcap_{i=1}^{n_{\mathsf{sig}}} \big\{C^{(i)} = c^{(i)}, Z^{(i)} = z^{(i)}, E^{(i)} = \widetilde{y}_\lambda{}^{(i)} \oplus \mathrm{bit}_\lambda(z^{(i)} - \langle c^{(i)}, x \rangle)\big\}\big)\,,\end{aligned}$$

such that Lemma 35 yields both assertions of (a). To show claim (b), we observe that $\mathbb{P}(C = c, Z = z, \widetilde{Y}_\lambda = b) = \mathbb{P}(C = c) \cdot \mathbb{P}(Z = z) \cdot \mathbb{P}\big(\widetilde{Y}_\lambda = b \mid C = c, Z = z\big)$ and

$$\begin{aligned}&\mathbb{P}\big(\widetilde{Y}_\lambda = b \mid C = c, Z = z\big) \\ &\quad = (1 - p_{\mathsf{err}})\, \mathbb{P}\big(b = \mathrm{bit}_\lambda(z - S) \mid C = c\big) + p_{\mathsf{err}}\, \mathbb{P}\big(b \neq \mathrm{bit}_\lambda(z - S) \mid C = c\big) \\ &\quad = (1 - p_{\mathsf{err}})\, \mathbb{P}\big(b = \mathrm{bit}_\lambda(z - S)\big) + p_{\mathsf{err}}\, \mathbb{P}\big(b \neq \mathrm{bit}_\lambda(z - S)\big)\,,\end{aligned}$$

where we used in the last step that C and S are independent according to Lemma 37 (a). □

A.2 Candidate keys and most likely keys

Recall from Section 4.1 that an element $x \in \mathcal{X}$ is said to satisfy the *bit constraint* imposed by a leaky signature $(c, z, \widetilde{y}_\lambda)$ if $\widetilde{y}_\lambda = \mathrm{bit}_\lambda(z - \langle c, x\rangle)$ (regardless of whether $\widetilde{y}_\lambda$ is the correct bit). The sets of candidate keys and most likely keys were defined as the sets of keys that satisfy all or a maximal set of bit constraints in a collection of leaky signatures, respectively. Below we provide a definition of these sets as images of a set-valued function, which is convenient for defining random versions of these sets later. Then we show that these sets represent optimal solutions to Problem 1 in the sense that they contain the solutions maximizing the attacker's success probability. Finally, we provide a characterization of bit constraints.

Definition 39. Let $n_{\mathsf{sig}} \geq 0$.

(a) We define the set-valued function $\Xi_{\mathsf{cand}} \colon (\mathcal{C} \times \mathbb{Z} \times \{0,1\})^{n_{\mathsf{sig}}} \to 2^{\mathcal{X}}$ by

$$\Xi_{\mathsf{cand}}\big((c^{(i)}, z^{(i)}, b^{(i)})_{i=1}^{n_{\mathsf{sig}}}\big) := \big\{x \in \mathcal{X} \mid b^{(i)} = \mathrm{bit}_\lambda(z^{(i)} - \langle c^{(i)}, x\rangle) \text{ for all } i \in [n_{\mathsf{sig}}]\big\}\,.$$

We refer to $\Xi_{\mathsf{cand}}\big((c^{(i)}, z^{(i)}, b^{(i)})_{i=1}^{n_{\mathsf{sig}}}\big)$ as the set of *candidate keys* given $(c^{(i)}, z^{(i)}, b^{(i)})_{i=1}^{n_{\mathsf{sig}}}$.

(b) We define the set-valued function $\Xi_{\mathsf{ml}} \colon (\mathcal{C} \times \mathbb{Z} \times \{0,1\})^{n_{\mathsf{sig}}} \to 2^{\mathcal{X}}$ by

$$\Xi_{\mathsf{ml}}\big((c^{(i)}, z^{(i)}, b^{(i)})_{i=1}^{n_{\mathsf{sig}}}\big) := \operatorname*{arg\,min}_{x \in \mathcal{X}} \#\big\{i \in [n_{\mathsf{sig}}] \mid b^{(i)} \neq \mathrm{bit}_\lambda(z^{(i)} - \langle c^{(i)}, x\rangle)\big\}\,.$$

We refer to $\Xi_{\mathsf{ml}}\big((c^{(i)}, z^{(i)}, b^{(i)})_{i=1}^{n_{\mathsf{sig}}}\big)$ as the set of *most likely keys* given $(c^{(i)}, z^{(i)}, b^{(i)})_{i=1}^{n_{\mathsf{sig}}}$.

Remark 40. For all $(c^{(i)}, z^{(i)}, b^{(i)})_{i=1}^{n_{\mathsf{sig}}} \in (\mathcal{C}\times\mathbb{Z}\times\{0,1\})^{n_{\mathsf{sig}}}$ with $\Xi_{\mathsf{cand}}\big((c^{(i)}, z^{(i)}, b^{(i)})_{i=1}^{n_{\mathsf{sig}}}\big) \neq \varnothing$, we have $\Xi_{\mathsf{cand}}\big((c^{(i)}, z^{(i)}, b^{(i)})_{i=1}^{n_{\mathsf{sig}}}\big) = \Xi_{\mathsf{ml}}\big((c^{(i)}, z^{(i)}, b^{(i)})_{i=1}^{n_{\mathsf{sig}}}\big)$.

Optimality with respect to the stochastic model

The following lemma shows that most likely keys according to Definition 39 are true to their name.

Lemma 41. *Let* $\mathcal{X}_{\mathsf{ml}} = \Xi_{\mathsf{ml}}\big((c^{(i)}, z^{(i)}, \widetilde{y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}}\big)$ *be the set of most likely keys given leaky signatures* $(c^{(i)}, z^{(i)}, \widetilde{y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}} \in \mathrm{supp}\big((C^{(i)}, Z^{(i)}, \widetilde{Y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}}\big)$. *Then*

$$\mathcal{X}_{\mathsf{ml}} = \operatorname*{arg\,max}_{x \in \mathcal{X}} \mathbb{P}\big(X^* = x \mid (C^{(i)}, Z^{(i)}, \widetilde{Y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}} = (c^{(i)}, z^{(i)}, \widetilde{y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}}\big) \tag{1}$$

$$= \operatorname*{arg\,max}_{x \in \mathcal{X}} \textstyle\prod_{i=1}^{n_{\mathsf{sig}}} \mathbb{P}\big(\widetilde{Y}_\lambda{}^{(i)} = \widetilde{y}_\lambda{}^{(i)} \mid X^* = x, C^{(i)} = c^{(i)}, Z^{(i)} = z^{(i)}\big) \tag{2}$$

$$= \operatorname*{arg\,max}_{x \in \mathcal{X}} \textstyle\prod_{i=1}^{n_{\mathsf{sig}}} \mathbb{P}\big(E = \widetilde{y}_\lambda{}^{(i)} \oplus \mathrm{bit}_\lambda(z^{(i)} - \langle c^{(i)}, x\rangle)\big)\,. \tag{3}$$

Proof. Using Lemma 38, the right-hand sides of (1) – (3) are seen to be equal. We have

$$\textstyle\prod_{i=1}^{n_{\mathsf{sig}}} \mathbb{P}\big(E = \widetilde{y}_\lambda{}^{(i)} \oplus \mathrm{bit}_\lambda(z^{(i)} - \langle c^{(i)}, x\rangle)\big) = (1 - p_{\mathsf{err}})^{n_{\mathsf{sig}} - w(x)} \cdot {p_{\mathsf{err}}}^{w(x)}\,, \tag{4}$$

where $w(x) := \#\big\{i \in [n_{\mathsf{sig}}] \mid \widetilde{y}_\lambda{}^{(i)} \neq \mathrm{bit}_\lambda(z^{(i)} - \langle c^{(i)}, x\rangle)\big\}$ for all $x \in \mathcal{X}$ and where the convention $0^0 = 1$ is used in the case $p_{\mathsf{err}} = 0$. Since $p_{\mathsf{err}} \in [0, \frac{1}{2})$ by assumption, we have $1 - p_{\mathsf{err}} > p_{\mathsf{err}}$. Therefore, the keys $x \in \mathcal{X}$ that maximize (4) are exactly those that minimize $w(x)$, which is exactly the definition of the set $\mathcal{X}_{\mathsf{ml}}$. □

Let $\mathcal{D} = \big\{\delta \colon (\mathcal{C} \times \mathcal{Z} \times \{0,1\})^{n_{\mathsf{sig}}} \to \mathcal{X}\big\}$ be the set of (deterministic) decision rules. A decision rule $\delta \in \mathcal{D}$ assigns a key guess $\widehat{x} := \delta\big((c^{(i)}, z^{(i)}, \widetilde{y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}}\big)$ to given leaky signatures $(c^{(i)}, z^{(i)}, \widetilde{y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}}$ and the key guess solves Problem 1 if $\widehat{x} = x^*$. The set of optimal decision rules is defined as

$$\mathcal{D}^* := \operatorname*{arg\,max}_{\delta \in \mathcal{D}} \mathbb{P}\big(X^* = \delta\big((C^{(i)}, Z^{(i)}, \widetilde{Y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}}\big)\big) \tag{5}$$

and contains all decision rules that maximize the success probability of solving Problem 1 (see [HRG14], where the term distinguishing rule is used). The following corollary shows that decision rules picking key guesses from the set of most likely keys are optimal.

Corollary 42. *The set of optimal decision rules defined in (5) is given by*

$$\mathcal{D}^* = \left\{\delta \in \mathcal{D} \mid \delta(t) \in \Xi_{\mathsf{ml}}(t) \text{ for all } t \in \operatorname{supp}\big((C^{(i)}, Z^{(i)}, \widetilde{Y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}}\big)\right\}.$$

Proof. Denote $T \coloneqq (C^{(i)}, Z^{(i)}, \widetilde{Y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}}$. The statement then follows from equation (1) in Lemma 41, since we have $\mathbb{P}(X^* = \delta(T)) = \sum_{t \in \operatorname{supp}(T)} \mathbb{P}\big(X^* = \delta(t) \mid T = t\big) \cdot \mathbb{P}(T = t)$. □

Characterization

Next, we derive a characterization for the bit constraints. We need the following fundamental properties of the function bit_j, which we state without proof.

Lemma 43. *Let $j \geq 0$ and let $y \in \mathbb{Z}$.*

(a) We have $\mathrm{bit}_j(y) = \mathrm{bit}_j(y + u \cdot 2^{j+1})$ for all $u \in \mathbb{Z}$.

(b) We have $\mathrm{bit}_j(y) = 1 - \mathrm{bit}_j(y \pm 2^j)$.

(c) We have $\mathrm{bit}_j(y) = 1 - \mathrm{bit}_j(-y - 1)$.

(d) We have $\mathrm{bit}_j(y) = 0$ if and only if $y \bmod^{\mp} 2^{j+1} \geq 0$.

(e) We have $y \bmod^{\mp} 2^{j+1} = y \bmod^{\mp} 2^j$ if and only if $\mathrm{bit}_j(y) = \mathrm{bit}_{j-1}(y)$. □

The next lemma characterizes the constraints that a leaky signature (c, z, y_λ) with exact leakage imposes on the scalar product $s = \langle c, x \rangle$. This provides a way to describe $\mathcal{X}_{\mathsf{cand}}$ as a set defined by linear constraints, which in Section 5 allows us to tackle Problem 1 via integer linear programming. Note that our notation is not compatible with [Dam+25a, Section 4]; in particular, $\overline{z}$ is defined differently.

Lemma 44. *Let $s \in \mathcal{S}$, let $z \in \mathbb{Z}$, and let $b \in \{0, 1\}$ be such that $b = \mathrm{bit}_\lambda(z - s)$. Define $\overline{z} \coloneqq z \bmod^{\mp} 2^\lambda \in [-2^{\lambda-1}, 2^{\lambda-1}) \cap \mathbb{Z}$.*

(a) There exists a unique $u \in \mathbb{Z}$ such that

$$\begin{cases} \overline{z} + 1 - 2^\lambda \leq s + u \cdot 2^{\lambda+1} \leq \overline{z} & \text{if } b \oplus \mathrm{bit}_\lambda(z) \oplus \mathrm{bit}_{\lambda-1}(z) = 0\,, \\ \overline{z} + 1 \qquad\; \leq s + u \cdot 2^{\lambda+1} \leq \overline{z} + 2^\lambda & \text{otherwise}. \end{cases}$$

We have $u = \lfloor (\overline{z} - s + 2^\lambda)/2^{\lambda+1} \rfloor \in [-\nu, \nu] \cap \mathbb{Z}$, where $\nu \coloneqq \lceil (\beta - 2^{\lambda-1})/2^{\lambda+1} \rceil$.

(b) If $\lambda \geq \log_2(\beta) + 1$, we have

$$\begin{cases} -\beta \leq s \leq \overline{z} & \text{if } b \oplus \mathrm{bit}_\lambda(z) \oplus \mathrm{bit}_{\lambda-1}(z) = 0\,, \\ \overline{z} + 1 \leq s \leq \beta & \text{otherwise}. \end{cases}$$

Proof. Replacing z with $z + 2^\lambda$, if necessary, we may assume $z \bmod^{\mp} 2^{\lambda+1} = \overline{z}$ and, equivalently by Lemma 43 (e), $\mathrm{bit}_\lambda(z) \oplus \mathrm{bit}_{\lambda-1}(z) = 0$. By Lemma 43 (a) and (b), this replacement does not change $\overline{z}$ and $\mathrm{bit}_{\lambda-1}(z)$, and inverts both b and $\mathrm{bit}_\lambda(z)$, hence the assertions of the lemma are not affected by this change.

There exists a unique $u \in \mathbb{Z}$ such that $\overline{z} - s - u \cdot 2^{\lambda+1} = (\overline{z} - s) \bmod^{\mp} 2^{\lambda+1} \in [-2^\lambda, 2^\lambda)$, which is given by $u = \lfloor (\overline{z} - s + 2^\lambda)/2^{\lambda+1} \rfloor \in [-\nu, \nu]$.

Since we arranged $z \bmod^{\mp} 2^{\lambda+1} = \overline{z}$, Lemma 43 (a) implies $\mathrm{bit}_\lambda(\overline{z} - s - u \cdot 2^{\lambda+1}) = \mathrm{bit}_\lambda(\overline{z} - s) = \mathrm{bit}_\lambda(z - s) = b$. By Lemma 43 (d), we obtain $\overline{z} - s - u \cdot 2^{\lambda+1} \in [0, 2^\lambda)$ if $b = 0$ and $\overline{z} - s - u \cdot 2^{\lambda+1} \in [-2^\lambda, 0)$ if $b = 1$. This implies claim (a).

Now assume $\lambda \geq \log_2(\beta) + 1$. Then we have $\nu = u = 0$ and $\overline{z} + 1 - 2^\lambda \leq -2^{\lambda-1} \leq -\beta \leq s \leq \beta \leq 2^{\lambda-1} \leq \overline{z} + 2^\lambda$. Therefore, claim (b) follows from (a). □

Remark 45. In the situation of Lemma 44, we have $u = 0$ if and only if $\overline{z} - s \in [-2^\lambda, 2^\lambda)$. These equivalent conditions are implied if $s \in [-2^{\lambda-1}, 2^{\lambda-1})$, which in turn is implied if $\lambda \geq \log_2(\beta) + 1$.

We will refer to parameters with $\lambda < \log_2(\beta) + 1$ as the *low-λ regime* and to parameters with $\lambda > \log_2(\beta) + 1$ as the *high-λ regime*.

Remark 46. In the definition of the high-λ regime, we require λ to be strictly larger than $\log_2(\beta) + 1$, because this condition implies further convenient properties such as the assertion of Lemma 54. The case $\lambda = \log_2(\beta) + 1$ is of no practical relevance, because β is not a power of 2 for all ML-DSA parameter sets.

The following characterization, a direct consequence of the previous lemma, will be used repeatedly for proofs in the high-λ regime.

Corollary 47. *Let $s, s^* \in \mathcal{S}$ and let $z \in \mathbb{Z}$ be such that $z \bmod^\mp 2^\lambda - s \in [-2^\lambda, 2^\lambda)$ and $z \bmod^\mp 2^\lambda - s^* \in [-2^\lambda, 2^\lambda)$ (which is always fulfilled if $\lambda \geq \log_2(\beta) + 1$). Then we have $\mathrm{bit}_\lambda(z - s) \neq \mathrm{bit}_\lambda(z - s^*)$ if and only if $s \leq z \bmod^\mp 2^\lambda < s^*$ or $s^* \leq z \bmod^\mp 2^\lambda < s$.*

Proof. This follows from Lemma 44. □

A.3 Mutual information of key and leakage

In this subsection, we investigate the amount of information leaky signatures contain about the key. We begin by quantifying the information that a fixed leaky signature, viewed as a realization of $(C, Z, \widetilde{Y}_\lambda)$, contains about the random key X^*. The notion of informative leaky signatures introduced in [Dam+25a] emerges as a special case. Finally, we adapt the stochastic model introduced in Section A.1 to the subfamily of informative leaky signatures and study properties of this restricted family.

Mutual information of key and leaky signature realizations

To quantify the amount of information a fixed leaky signature $(c, z, b) \in \mathrm{supp}\big(C, Z, \widetilde{Y}_\lambda\big)$ contains about the unknown key x^* on average, we consider the quantity

$$\mathrm{I}\big(X^*; C = c, Z = z, \widetilde{Y}_\lambda = b\big) := \mathrm{H}\big(X^*\big) - \mathrm{H}\big(X^* \mid C = c, Z = z, \widetilde{Y}_\lambda = b\big)\,,$$

where $\mathrm{H}\big(X^*\big)$ and $\mathrm{H}\big(X^* \mid C = c, Z = z, \widetilde{Y}_\lambda = b\big)$ denote the Shannon entropies of the random variables X^* and $\big(X^* \mid C = c, Z = z, \widetilde{Y}_\lambda = b\big)$, respectively.

Lemma 48. *Let $(c, z, b) \in \mathrm{supp}\big(C, Z, \widetilde{Y}_\lambda\big)$, and let us abbreviate $p_{z,b} := \mathbb{P}\big(b = \mathrm{bit}_\lambda(z - S)\big)$ and $q_{z,b} := (1 - p_{\mathsf{err}})p_{z,b} + p_{\mathsf{err}}p_{z,1-b}$, where $S = \langle C, X^* \rangle$.*

(a) We have $\mathrm{I}\big(X^*; C = c, Z = z, \widetilde{Y}_\lambda = b\big) = I(z, b)$, where

$$I(z, b) := -\log_2\big(q_{z,b}\big) + \frac{(1 - p_{\mathsf{err}})\log_2(1 - p_{\mathsf{err}})p_{z,b} + p_{\mathsf{err}}\log_2(p_{\mathsf{err}})p_{z,1-b}}{q_{z,b}}\,,$$

and we use the convention $0 \cdot \log_2(0) = 0$ when $p_{\mathsf{err}} = 0$.

In particular, if $p_{\mathsf{err}} = 0$, then $I(z, b) = -\log_2(p_{z,b}) = \log_2(\#\mathcal{X}) - \log_2(\#\Xi_{\mathsf{cand}}(c, z, b))$.

(b) We have $I(z, b) \geq 0$, with equality if and only if $p_{z,0} \in \{0, 1\}$.

(c) We have $I(z, b) > 0$ if and only if $z \bmod^\mp 2^\lambda \in [-\beta, \beta)$. If $\lambda \leq \log_2(\beta) + 1$, then both equivalent conditions are always fulfilled.

Proof. By Lemma 38 (a) and (b), we have

$$\mathbb{P}\big(X^* = x \mid C = c, Z = z, \widetilde{Y}_\lambda = b\big) = \frac{\mathbb{P}\left(E = b \oplus \mathrm{bit}_\lambda(z - \langle c, x\rangle)\right)}{(1 - p_{\mathsf{err}})p_{z,b} + p_{\mathsf{err}}p_{z,1-b}} \cdot \mathbb{P}\big(X^* = x\big)\,. \tag{6}$$

This identity combined with the observation that

$$\begin{aligned}
&\textstyle\sum_{x\in\mathcal{X}} \mathbb{P}\left(E = b \oplus \mathrm{bit}_\lambda(z - \langle c, x\rangle)\right) \cdot \mathbb{P}(X^* = x) \cdot \log_2\big(\mathbb{P}(E = b \oplus \mathrm{bit}_\lambda(z - \langle c, x\rangle))\big) \\
&\quad = \textstyle\sum_{x\in\mathcal{X}} \sum_{e\in\{0,1\}} 1_{e = b\oplus \mathrm{bit}_\lambda(z-\langle c,x\rangle)} \cdot \mathbb{P}(E = e) \cdot \log_2(\mathbb{P}(E = e)) \cdot \mathbb{P}(X^* = x) \\
&\quad = \textstyle\sum_{e\in\{0,1\}} \mathbb{E}\big(1_{e = b\oplus \mathrm{bit}_\lambda(z-\langle c,X^*\rangle)} \cdot \mathbb{P}(E = e) \cdot \log_2(\mathbb{P}(E = e))\big)
\end{aligned}$$

yields part (a).

Because the random variable $\big(X^* \mid C = c, Z = z, \widetilde{Y}_\lambda = b\big)$ takes values in $\mathcal{X}$, we have that $\mathrm{H}\big(X^* \mid C = c, Z = z, \widetilde{Y}_\lambda = b\big) \le \log_2\big(\#\mathcal{X}\big) = \mathrm{H}\big(X^*\big)$, with equality if and only if $\big(X^* \mid C = c, Z = z, \widetilde{Y}_\lambda = b\big)$ is uniformly distributed on $\mathcal{X}$ (see [CT06, Theorem 2.4.6]). This implies $I(z, b) \ge 0$. If $I(z, b) = 0$, then (6) implies that $\mathbb{P}\big(E = b \oplus \mathrm{bit}_\lambda(z - \langle c, x\rangle)\big) = (1 - p_{\mathsf{err}})p_{z,b} + p_{\mathsf{err}}p_{z,1-b}$ for all $x \in \mathcal{X}$. However, since $\mathbb{P}\big(E = b \oplus \mathrm{bit}_\lambda(z - \langle c, x\rangle)\big) \in \{1 - p_{\mathsf{err}}, p_{\mathsf{err}}\}$, we obtain $p_{z,b}, p_{z,1-b} \in \{0, 1\}$. Conversely, if $p_{z,0} \in \{0, 1\}$ (if $p_{\mathsf{err}} = 0$, this implies $p_{z,b} = 1$ by assumption), we obtain $I(z, b) = 0$ by substitution. This concludes the proof of (b).

As a consequence of (b), we know that $I(z, b) > 0$ if and only if $p_{z,0} \in (0, 1)$. The latter holds if and only if there exist $s_1, s_2 \in \mathcal{S}$ such that $\mathrm{bit}_\lambda(z - s_1) \ne \mathrm{bit}_\lambda(z - s_2)$. If $\lambda \le \log_2(\beta) + 1$, then the latter is always fulfilled with $s_1 = -\beta$ and $s_2 = -\beta + 2^\lambda$, and so is the condition $z \bmod^\mp 2^\lambda \in [-\beta, \beta)$. Now let $\lambda > \log_2(\beta) + 1$. If there exist $s_1, s_2 \in \mathcal{S}$ with $s_1 < s_2$ (say) such that $\mathrm{bit}_\lambda(z - s_1) \ne \mathrm{bit}_\lambda(z - s_2)$, then Corollary 47 implies $z \bmod^\mp 2^\lambda \in [s_1, s_2) \subseteq [-\beta, \beta)$. Conversely, if $z \bmod^\mp 2^\lambda \in [-\beta, \beta)$, then Corollary 47 applied with $s_1 = -\beta$ and $s_2 = \beta$ implies $\mathrm{bit}_\lambda(z - s_1) \ne \mathrm{bit}_\lambda(z - s_2)$. This finishes the proof of (c). □

Informative leaky signatures

We say that a leaky signature $(c, z, b) \in \mathrm{supp}\big(C, Z, \widetilde{Y}_\lambda\big)$ is *informative* if $I(z, b) > 0$. This definition basically coincides with the notion of the same name introduced in [Dam+25a]. By Lemma 48 (c), (c, z, b) is informative if and only if $z \in [-\beta, \beta) + 2^\lambda\mathbb{Z}$, hence the informativeness of (c, z, b) only depends on z.

Lemma 49. *Define* $\mathcal{Z}_{\mathsf{inf}} \coloneqq \mathcal{Z} \cap \big([-\beta, \beta) + 2^\lambda\mathbb{Z}\big)$ *and* $p_{\mathsf{inf}} \coloneqq \mathbb{P}(Z \in \mathcal{Z}_{\mathsf{inf}})$.

(a) We have

$$\mathcal{Z}_{\mathsf{inf}} = \begin{cases} \mathcal{Z} & \text{if } 0 \le \lambda \le \log_2(\beta) + 1\,, \\ \bigcup_{j=-\gamma/2^\lambda+1}^{\gamma/2^\lambda-1} [j2^\lambda - \beta, j2^\lambda + \beta) \cap \mathbb{Z} & \text{if } \log_2(\beta) + 1 < \lambda \le \log_2(\gamma)\,, \\ [-\beta, \beta) \cap \mathbb{Z} & \text{if } \lambda \ge \log_2(\gamma)\,, \end{cases}$$

where the union in the middle case is disjoint.

(b) We have

$$p_{\mathsf{inf}} = \begin{cases} 1 & \text{if } 0 \le \lambda \le \log_2(\beta) + 1\,, \\ \frac{2\beta(\gamma/2^{\lambda-1} - 1)}{2(\gamma-\beta)-1} & \text{if } \log_2(\beta) + 1 < \lambda \le \log_2(\gamma)\,, \\ \frac{2\beta}{2(\gamma-\beta)-1} & \text{if } \lambda \ge \log_2(\gamma)\,. \end{cases}$$

Proof. If $0 \le \lambda \le \log_2(\beta) + 1$, then $[-\beta, \beta) + 2^\lambda\mathbb{Z} = \mathbb{R}$, hence $\mathcal{Z}_{\mathsf{inf}} = \mathcal{Z}$. If $\lambda > \log_2(\beta) + 1$, then the intervals $\{[j2^\lambda - \beta, j2^\lambda + \beta)\}_{j\in\mathbb{Z}}$ are disjoint and, using that γ is a power

of 2 and $\gamma > 2\beta$ (see Remark 6), we obtain $[j2^\lambda - \beta, j2^\lambda + \beta) \subseteq (-\gamma + \beta, \gamma - \beta)$ for all $\lfloor -\gamma/2^\lambda \rfloor + 1 \leq j \leq \lceil \gamma/2^\lambda \rceil - 1$ and $[j2^\lambda - \beta, j2^\lambda + \beta) \cap (-\gamma + \beta, \gamma - \beta) = \varnothing$ for all $j \leq \lfloor -\gamma/2^\lambda \rfloor$ and $j \geq \lceil \gamma/2^\lambda \rceil$. We get

$$\begin{aligned} \mathcal{Z}_{\mathsf{inf}} &= (-\gamma + \beta, \gamma - \beta) \cap \mathbb{Z} \cap \textstyle\bigcup_{j \in \mathbb{Z}} [j2^\lambda - \beta, j2^\lambda + \beta) \\ &= \textstyle\bigcup_{j=\lfloor -\gamma/2^\lambda \rfloor + 1}^{\lceil \gamma/2^\lambda \rceil - 1} [j2^\lambda - \beta, j2^\lambda + \beta) \cap \mathbb{Z}\,. \end{aligned}$$

This finishes the proof of claim (a), which readily implies (b) as $p_{\mathsf{inf}} = \#\mathcal{Z}_{\mathsf{inf}}/\#\mathcal{Z}$. □

Remark 50. For $\log_2(\beta) + 1 < \lambda \ll \log_2(\gamma)$, we have the approximation $p_{\mathsf{inf}} \approx \frac{\beta}{2^{\lambda-1}}$, and for $\lambda = \log_2(\gamma)$, we have the approximation $p_{\mathsf{inf}} \approx \frac{\beta}{2^\lambda} = \frac{\beta}{\gamma}$. This is in accordance with the empirical observations in [Dam+25a, Section 6].

Restriction to informative leaky signatures

Definition 51. Define $\iota(z) \coloneqq 1_{[-\beta,\beta)+2^\lambda\mathbb{Z}}(z)$ for $z \in \mathbb{R}$. Define the random variables

$$I(h) \coloneqq \inf\big\{ i \geq h \mid \iota(Z^{(1)}) + \cdots + \iota(Z^{(i)}) = h \big\}\,, \qquad h \geq 0\,.$$

Denoting by W any of C, Z, Y, Y_λ, E, $\widetilde{Y}_\lambda$, we introduce the notation ${W_{\mathsf{inf}}}^{(h)} \coloneqq W^{(I(h))}$ for all $h \geq 1$. If just a single index $h \geq 1$ is under consideration, we omit the index "(h)".

Lemma 52. *The random variables in Definition 51 satisfy the following properties.*

(a) The family $\{X^*\} \cup \{{C_{\mathsf{inf}}}^{(h)}, {Z_{\mathsf{inf}}}^{(h)}, {E_{\mathsf{inf}}}^{(h)}, I(h) - I(h-1)\}_{h \geq 1}$ of random variables is independent.

(b) The random variables $\{{C_{\mathsf{inf}}}^{(h)}\}_{h \geq 1}$ are uniformly distributed on $\mathcal{C}$.

(c) The random variables $\{{Z_{\mathsf{inf}}}^{(h)}\}_{h \geq 1}$ are uniformly distributed on $\mathcal{Z}_{\mathsf{inf}}$.

(d) The random variables $\{{E_{\mathsf{inf}}}^{(h)}\}_{h \geq 1}$ are Bernoulli-distributed with $\mathbb{P}(E_{\mathsf{inf}} = 1) = p_{\mathsf{err}}$.

(e) The random variables $\{I(h) - I(h-1)\}_{h \geq 1}$ are geometrically distributed with support $\mathbb{N}_{\geq 1}$ and success probability $\mathbb{P}(\iota(Z) = 1) = p_{\mathsf{inf}}$.

Proof. Set $D(h) \coloneqq I(h) - I(h-1)$ for $h \geq 1$. Let $n \geq 1$ and let $d_1, \ldots, d_n \in \mathbb{N}_{\geq 1}$. Set $i_0 \coloneqq 0$ and set $i_h \coloneqq d_1 + \cdots + d_h$ for $h \in [n]$. We have

$$\begin{aligned} \textstyle\bigcap_{h=1}^n \big\{ D(h) = d_h \big\} &= \textstyle\bigcap_{h=1}^n \big\{ I(h) = i_h \big\} \\ &= \textstyle\bigcap_{h=1}^n \big\{ \iota(Z^{(i_{h-1}+1)}) = \cdots = \iota(Z^{(i_h-1)}) = 0, \iota(Z^{(i_h)}) = 1 \big\}\,. \end{aligned}$$

Observing that $\mathbb{P}\big(Z = z, \iota(Z) = 1\big) = \frac{\iota(z)}{\#\mathcal{Z}} = \frac{\iota(z)}{\#\mathcal{Z}_{\mathsf{inf}}} \cdot p_{\mathsf{inf}}$ for $z \in \mathcal{Z}$ and using the independence of the family $\{X^*\} \cup \{C^{(i)}, Z^{(i)}, E^{(i)}\}_{i \geq 1}$, we obtain

$$\begin{aligned} &\mathbb{P}\big(\{X^* = x\} \cap \textstyle\bigcap_{h=1}^n \big\{ {C_{\mathsf{inf}}}^{(h)} = c^{(h)}, {Z_{\mathsf{inf}}}^{(h)} = z^{(h)}, {E_{\mathsf{inf}}}^{(h)} = e^{(h)}, D(h) = d_h \big\}\big) \\ &\quad = \mathbb{P}\big(X^* = x\big) \cdot \mathbb{P}\big(\textstyle\bigcap_{h=1}^n \big\{ C^{(i_h)} = c^{(h)}, Z^{(i_h)} = z^{(h)}, E^{(i_h)} = e^{(h)}, I(h) = i_h \big\}\big) \\ &\quad = \mathbb{P}\big(X^* = x\big) \cdot \textstyle\prod_{h=1}^n \big(\mathbb{P}(C = c^{(h)}) \cdot \mathbb{P}(Z = z^{(h)}, \iota(Z) = 1) \cdot \mathbb{P}(E = e^{(h)}) \cdot (1 - p_{\mathsf{inf}})^{d_h - 1}\big) \\ &\quad = \mathbb{P}\big(X^* = x\big) \cdot \textstyle\prod_{h=1}^n \big(\mathbb{P}(C = c^{(h)}) \cdot \frac{\iota(z^{(h)})}{\#\mathcal{Z}_{\mathsf{inf}}} \cdot \mathbb{P}(E = e^{(h)}) \cdot (1 - p_{\mathsf{inf}})^{d_h - 1} p_{\mathsf{inf}}\big)\,, \end{aligned}$$

which implies (a) – (e). □

Number of informative leaky signatures

The following lemma shows (a) how many informative signatures are contained in a given set of leaky signatures on average and (b) how many leaky signatures have to be generated on average to obtain a target number of informative signatures.

Lemma 53. *Let $p_{\mathsf{inf}} = \mathbb{P}(Z \in \mathcal{Z}_{\mathsf{inf}})$ be as defined in Lemma 49.*

(a) For $n_{\mathsf{sig}} \geq 0$ let $N_{\mathsf{inf}} := \iota(Z^{(1)}) + \cdots + \iota(Z^{(n_{\mathsf{sig}})})$ be the random number of informative leaky signatures in $(C^{(i)}, Z^{(i)}, \widetilde{Y}_\lambda{}^{(i)})_{i=1}^{n_{\mathsf{sig}}}$. Then, $\mathbb{E}(N_{\mathsf{inf}}) = n_{\mathsf{sig}} \cdot p_{\mathsf{inf}}$.

(b) For $n_{\mathsf{inf}} \geq 0$ let $N_{\mathsf{sig}} := I(n_{\mathsf{inf}})$ be the smallest random number with $(C^{(i)}, Z^{(i)}, \widetilde{Y}_\lambda{}^{(i)})_{i=1}^{N_{\mathsf{sig}}}$ containing n_{inf} informative leaky signatures. Then, $\mathbb{E}(N_{\mathsf{sig}}) = n_{\mathsf{inf}}/p_{\mathsf{inf}}$.

Proof. The random variable N_{inf} is the sum of n_{sig} Bernoulli random variables with success probability p_{inf} and the random variable N_{sig} is the sum of n_{inf} geometrically distributed random variables with support $\mathbb{N}_{\geq 1}$ and success probability p_{inf}. Therefore, both claims follow by linearity of expectation. □

Uniform hardness in the high-λ regime

We next show that, with the same number of informative signatures available, random instances of Problem 1 are equally hard to solve for all choices of λ in the high-λ regime. This justifies focusing henceforth on $\lambda = 8$ for ML-DSA-44 and ML-DSA-87, and on $\lambda = 9$ for ML-DSA-65, as the representative choices for λ in the high-λ regime.

Lemma 54. *If $\lambda > \log_2(\beta) + 1$, the random variables $\{Z_{\mathsf{inf}}{}^{(i)} \bmod^{\mp} 2^\lambda\}_{i \geq 1}$ are uniformly distributed on $[-\beta, \beta) \cap \mathbb{Z}$. In particular, their distributions are independent of λ.*

Proof. By Lemma 49 (a), $\mathcal{Z}_{\mathsf{inf}}$ can be written as a disjoint union $\bigcup_{j \in \mathcal{J}} [j2^\lambda - \beta, j2^\lambda + \beta) \cap \mathbb{Z}$ indexed by some set $\mathcal{J}$. In combination with Lemma 52 (c) this implies that, for every $z \in [-\beta, \beta) \cap \mathbb{Z}$, we have $\mathbb{P}(Z_{\mathsf{inf}} \bmod^{\mp} 2^\lambda = z) = \sum_{j \in \mathcal{J}} \mathbb{P}(Z_{\mathsf{inf}} = z - j2^\lambda) = \frac{\#\mathcal{J}}{\#\mathcal{Z}_{\mathsf{inf}}} = \frac{1}{2\beta}$. □

Lemma 55. *Let $B^{(i)}(x)$ be the event $\{\widetilde{Y}_{\lambda,\mathsf{inf}}{}^{(i)} \neq \mathrm{bit}_\lambda(Z_{\mathsf{inf}}{}^{(i)} - \langle C_{\mathsf{inf}}{}^{(i)}, x\rangle)\}$ for $i \geq 1$. If $\lambda > \log_2(\beta) + 1$, then, for each $x \in \mathcal{X}$, the distribution of $(1_{B^{(i)}(x)})_{i \geq 1}$ is independent of λ, and, in particular, so is the distribution of the random set of most likely keys $\Xi_{\mathsf{ml}}\big((C_{\mathsf{inf}}{}^{(i)}, Z_{\mathsf{inf}}{}^{(i)}, \widetilde{Y}_{\lambda,\mathsf{inf}}{}^{(i)})_{i=1}^{n_{\mathsf{inf}}}\big)$.*

Proof. For simplicity, we only prove that $\mathbb{P}\big(\widetilde{Y}_{\lambda,\mathsf{inf}} \neq \mathrm{bit}_\lambda(Z_{\mathsf{inf}} - \langle C_{\mathsf{inf}}, x\rangle)\big)$ is independent of λ. The more general claim about the distribution of $(1_{B^{(i)}(x)})_{i \geq 1}$ follows similarly. Defining $f(x, x^*) = \mathbb{P}(\langle C_{\mathsf{inf}}, x^*\rangle \leq Z_{\mathsf{inf}} \bmod^{\mp} 2^\lambda < \langle C_{\mathsf{inf}}, x\rangle)$, we have by Corollary 47

$$\mathbb{P}(Y_{\lambda,\mathsf{inf}} \neq \mathrm{bit}_\lambda(Z_{\mathsf{inf}} - \langle C_{\mathsf{inf}}, x\rangle)) = f(x, X^*) + f(X^*, x)\,.$$

Lemma 54 shows that this probability is independent of λ. Then, the same is true for the probability $\mathbb{P}\big(\widetilde{Y}_{\lambda,\mathsf{inf}} \neq \mathrm{bit}_\lambda(Z_{\mathsf{inf}} - \langle C_{\mathsf{inf}}, x\rangle)\big)$, since E_{inf} is independent of X^*, C_{inf}, Z_{inf}. □

A.4 Expected number of candidate keys

In this subsection, we derive lower and upper bounds for the expected number of elements in the random set

$$\mathfrak{X}_{\mathsf{cand,inf}} := \Xi_{\mathsf{cand}}\big((C_{\mathsf{inf}}{}^{(i)}, Z_{\mathsf{inf}}{}^{(i)}, Y_{\lambda,\mathsf{inf}}{}^{(i)})_{i=1}^{n_{\mathsf{inf}}}\big)$$

of candidate keys compatible with n_{inf} randomly sampled informative signatures with exact leakage ($p_{\mathsf{err}} = 0$). The primary goal is to estimate how many leaky signatures are typically required before we can reasonably expect that only one key could have generated them.

Remark 56. The results in this section will be formulated for informative signatures, i.e., in terms of n_{inf} and the random variables C_{inf}, Z_{inf}, $Y_{\lambda,\mathsf{inf}}$ etc. Analogous results hold without the restriction to the informativeness of the signatures, i.e., when considering n_{sig} and the random variables C, Z, Y_λ etc., instead. Namely, Lemma 57 can be adapted in the obvious way to this situation, whereas in Lemma 59, Lemma 60, and Lemma 61, the factor $\frac{1}{2\beta}$ must be replaced with $\frac{p_{\mathsf{inf}}}{2\beta}$.

Probability that two keys generate the same leaky signatures

Let X be an independent random key, i.e. a random variable uniformly distributed on $\mathcal{X}$ such that the family $\{X, X^*\} \cup \{{C_{\mathsf{inf}}}^{(i)}, {Z_{\mathsf{inf}}}^{(i)}\}_{i\geq 1}$ is independent. We denote by $A(n_{\mathsf{inf}})$ the event in which the first n_{inf} leaky signatures, derived from $({C_{\mathsf{inf}}}^{(i)}, {Z_{\mathsf{inf}}}^{(i)})_{i\geq 1}$, coincide for the key X and the true key X^*. More precisely, we define the event

$$A(n_{\mathsf{inf}}) \coloneqq \bigcap_{i=1}^{n_{\mathsf{inf}}} \Big\{ \mathrm{bit}_\lambda({Z_{\mathsf{inf}}}^{(i)} - \langle {C_{\mathsf{inf}}}^{(i)}, X^* \rangle) = \mathrm{bit}_\lambda({Z_{\mathsf{inf}}}^{(i)} - \langle {C_{\mathsf{inf}}}^{(i)}, X \rangle) \Big\} .$$

Not unexpectedly, the probability of this event is directly linked to the expected number of candidate keys, as stated in the next lemma.

Lemma 57. *We have* $\mathbb{E}(\#\mathfrak{X}_{\mathsf{cand,inf}}) = \#\mathcal{X} \cdot \mathbb{P}(A(n_{\mathsf{inf}}))$.

Proof. The identity follows from the computation

$$\begin{aligned}
&\mathbb{E}(\#\mathfrak{X}_{\mathsf{cand,inf}}) \\
&\quad = \textstyle\sum_{x\in\mathcal{X}} \mathbb{P}\big({Y_{\lambda,\mathsf{inf}}}^{(i)} = \mathrm{bit}_\lambda({Z_{\mathsf{inf}}}^{(i)} - \langle {C_{\mathsf{inf}}}^{(i)}, x \rangle) \text{ for all } i \in [n_{\mathsf{inf}}]\big) \\
&\quad = \textstyle\sum_{x\in\mathcal{X}} \mathbb{P}(A(n_{\mathsf{inf}}) \mid X = x) = \sum_{x\in\mathcal{X}} \mathbb{P}(A(n_{\mathsf{inf}}), X = x) \cdot \#\mathcal{X} = \mathbb{P}(A(n_{\mathsf{inf}})) \cdot \#\mathcal{X} ,
\end{aligned}$$

where we used in the penultimate step that X is uniformly distributed on $\mathcal{X}$. □

Remark 58. In general, $\mathbb{E}(\#\mathfrak{X}_{\mathsf{cand,inf}}) > 1$. Indeed, if $\eta > 0$, we have

$$\begin{aligned}
&\mathbb{P}\big(A(n_{\mathsf{inf}}), X^* = X\big) = \mathbb{P}\big(X^* = X\big) = 1/\#\mathcal{X} \text{ and} \\
&\mathbb{P}\big(A(n_{\mathsf{inf}}), B\big) = \mathbb{P}(B) > 0 ,
\end{aligned}$$

where $B = \big\{X_1^* = X_2 \neq X_1 = X_2^*\big\} \cap \bigcap_{j=3}^{256} \big\{X_j^* = X_j\big\} \cap \bigcap_{i=1}^{n_{\mathsf{inf}}} \big\{({C_{\mathsf{inf}}}^{(i)})_1 = ({C_{\mathsf{inf}}}^{(i)})_2\big\}$.

The formula in the next lemma will facilitate the estimation (or explicit computation) of the number $\mathbb{E}(\#\mathfrak{X}_{\mathsf{cand,inf}})$.

Lemma 59. *Assuming* $\lambda > \log_2(\beta) + 1$, *we have*

$$\mathbb{E}(\#\mathfrak{X}_{\mathsf{cand,inf}}) = \Theta(n_{\mathsf{inf}}) ,$$

where $\Theta(n_{\mathsf{inf}}) = \#\mathcal{X} \cdot \mathbb{E}(\theta(n_{\mathsf{inf}}, X - X^*))$ *with* $\theta(n_{\mathsf{inf}}, t) = \big(1 - \frac{1}{2\beta}\,\mathbb{E}(|\langle C_{\mathsf{inf}}, t\rangle|)\big)^{n_{\mathsf{inf}}}$. *Indeed, for all* $t \in \{x - x^* \mid x, x^* \in \mathcal{X}\}$, *we have* $\mathbb{P}\big(A(n_{\mathsf{inf}}) \mid X - X^* = t\big) = \theta(n_{\mathsf{inf}}, t)$.

Proof. Proving the last statement is sufficient, because the first follows from it by Lemma 57. Let $x, x^* \in \mathcal{X}$. We consider the event $B = \{X = x, X^* = x^*\}$. Clearly,

$$\mathbb{P}(A(n_{\mathsf{inf}}) \mid B) = \mathbb{P}\big(Y_{\lambda,\mathsf{inf}} = \mathrm{bit}_\lambda(Z_{\mathsf{inf}} - \langle C_{\mathsf{inf}}, X\rangle) \mid B\big)^{n_{\mathsf{inf}}}$$

due to the independence of the family $\{X, X^*\} \cup \{{C_{\mathsf{inf}}}^{(i)}, {Z_{\mathsf{inf}}}^{(i)}\}_{i\geq 1}$. In the high-$\lambda$ regime, defining $f(x, x^*) = \mathbb{P}(\langle C_{\mathsf{inf}}, x^*\rangle \leq Z_{\mathsf{inf}} \bmod^{\mp} 2^\lambda < \langle C_{\mathsf{inf}}, x\rangle)$, we have by Corollary 47

$$\mathbb{P}(Y_{\lambda,\mathsf{inf}} \neq \mathrm{bit}_\lambda(Z_{\mathsf{inf}} - \langle C_{\mathsf{inf}}, X\rangle) \mid B) = f(x, x^*) + f(x^*, x) .$$

Using that $Z_{\mathsf{inf}} \bmod^{\mp} 2^\lambda$ is uniformly distributed on $[-\beta, \beta) \cap \mathbb{Z}$ according to Lemma 54 and that $\langle C_{\mathsf{inf}}, x\rangle, \langle C_{\mathsf{inf}}, x^*\rangle$ take values in $[-\beta, \beta]$, we get

$$\begin{aligned} f(x, x^*) &= \tfrac{1}{2\beta} \textstyle\sum_{z\in\mathbb{Z}} \mathbb{P}\big(\langle C_{\mathsf{inf}}, x^*\rangle \le z < \langle C_{\mathsf{inf}}, x\rangle\big) \\ &= \tfrac{1}{2\beta}\, \mathbb{E}\left(\textstyle\sum_{z\in\mathbb{Z}} 1_{\langle C_{\mathsf{inf}}, x^*\rangle \le z < \langle C_{\mathsf{inf}}, x\rangle}\right) = \tfrac{1}{2\beta}\, \mathbb{E}(\max(0, \langle C_{\mathsf{inf}}, t\rangle)) = \tfrac{1}{4\beta}\, \mathbb{E}(|\langle C_{\mathsf{inf}}, t\rangle|)\,, \end{aligned}$$

where $t = x - x^*$. Similarly, it follows that $f(x^*, x) = \frac{1}{4\beta}\, \mathbb{E}(|\langle C_{\mathsf{inf}}, -t\rangle|) = \frac{1}{4\beta}\, \mathbb{E}(|\langle C_{\mathsf{inf}}, t\rangle|)$. Consequently,

$$\mathbb{P}(Y_{\lambda,\mathsf{inf}} \neq \mathrm{bit}_\lambda(Z_{\mathsf{inf}} - \langle C_{\mathsf{inf}}, X\rangle) \mid B) = \tfrac{1}{2\beta}\, \mathbb{E}(|\langle C_{\mathsf{inf}}, t\rangle|)\,,$$

by which we obtain the desired identity $\mathbb{P}(A(n_{\mathsf{inf}}) \mid X - X^* = t) = \theta(n_{\mathsf{inf}}, t)$. □

Lower bound

Lemma 60. *Assuming $\lambda > \log_2(\beta) + 1$, we have*

$$\mathbb{E}(\#\mathfrak{X}_{\mathsf{cand},\mathsf{inf}}) \ge \Theta_{\mathsf{lower}}(n_{\mathsf{inf}})\,,$$

where $\Theta_{\mathsf{lower}}(n_{\mathsf{inf}}) = \#\mathcal{X} \cdot \mathbb{E}(\theta_{\mathsf{lower}}(n_{\mathsf{inf}}, X - X^*))$ with $\theta_{\mathsf{lower}}(n_{\mathsf{inf}}, t) = \big(1 - \frac{1}{2\beta}\sqrt{\frac{\tau}{256}}\|t\|_2\big)^{n_{\mathsf{inf}}}$. For the ML-DSA parameter sets, the lower bound Θ_{lower} is shown in Figure 4.

Proof. By Lemma 36, $\mathbb{E}(\langle C_{\mathsf{inf}}, t\rangle^2 \mid \mathfrak{J}) = \sum_{j\in\mathfrak{J}} t_j^2$ and $\mathbb{P}(j \in \mathfrak{J}) = \frac{\tau}{256}$ for all $j \in [256]$, so

$$\mathbb{E}(\langle C_{\mathsf{inf}}, t\rangle^2) = \mathbb{E}\big(\textstyle\sum_{j\in\mathfrak{J}} t_j^2\big) = \tfrac{\tau}{256}\|t\|_2^2\,.$$

Hence, $\frac{\tau}{256}\|t\|_2^2 - \mathbb{E}(|\langle C_{\mathsf{inf}}, t\rangle|)^2 = \mathrm{Var}(|\langle C_{\mathsf{inf}}, t\rangle|) \ge 0$, i.e. $\frac{1}{2\beta}\, \mathbb{E}(|\langle C_{\mathsf{inf}}, t\rangle|) \le \frac{1}{2\beta}\sqrt{\frac{\tau}{256}}\|t\|_2$, so

$$\big(1 - \tfrac{1}{2\beta}\sqrt{\tfrac{\tau}{256}}\|X - X^*\|_2\big)^{n_{\mathsf{inf}}} \le \big(1 - \tfrac{1}{2\beta}\, \mathbb{E}(|\langle C_{\mathsf{inf}}, X - X^*\rangle|)\big)^{n_{\mathsf{inf}}} = \theta(n_{\mathsf{inf}}, X - X^*)\,.$$

Taking expectations, and applying Lemma 59, completes the proof. □

Upper bound

Lemma 61. *Assuming $\lambda > \log_2(\beta) + 1$, we have*

$$\mathbb{E}(\#\mathfrak{X}_{\mathsf{cand},\mathsf{inf}}) \le \Theta_{\mathsf{upper}}(n_{\mathsf{inf}})\,,$$

where $\Theta_{\mathsf{upper}}(n_{\mathsf{inf}}) = \#\mathcal{X} \cdot \mathbb{E}(\theta_{\mathsf{upper}}(n_{\mathsf{inf}}, X - X^*))$ with $\theta_{\mathsf{upper}}(n_{\mathsf{inf}}, 0) = 1$ and, for $t \neq 0$,

$$\theta_{\mathsf{upper}}(n_{\mathsf{inf}}, t) = \big(1 - \tfrac{1}{2\beta}\tfrac{1}{\sqrt{2}}\sqrt{\tfrac{\tau}{256}}\|t\|_2 \cdot \big(1 + \tfrac{256-\tau}{255\tau}(256(\|t\|_4/\|t\|_2)^4 - 1)\big)^{-1/2}\big)^{n_{\mathsf{inf}}}\,.$$

For the ML-DSA parameter sets, the upper bound Θ_{upper} is shown in Figure 4.

Proof. We consider the random variable $T := \mathbb{E}(\langle C_{\mathsf{inf}}, t\rangle^2 \mid \mathfrak{J}) = \sum_{j\in\mathfrak{J}} t_j^2$, for which, in the proof of Lemma 60, it was shown that $\mathbb{E}(T) = \frac{\tau}{256}\|t\|_2^2$. We will also need the identity

$$\mathbb{E}(T^2) = \mathbb{E}\big(\textstyle\sum_{j,k\in\mathfrak{J}} t_j^2 t_k^2\big) = \sum_{j,k\in[256]} \mathbb{P}(j, k \in \mathfrak{J})\, t_j^2 t_k^2 = \tfrac{\tau}{256}\|t\|_4^4 + \tfrac{\tau(\tau-1)}{256\cdot 255}(\|t\|_2^4 - \|t\|_4^4)\,,$$

where in the last step we used the relation $\sum_{j\neq k\in[256]} t_j^2 t_k^2 = \|t\|_2^4 - \|t\|_4^4$ and the fact that the indices in $\mathfrak{J}$ are chosen uniformly according to Lemma 36. It follows that, for $t \neq 0$,

$$\mathbb{E}(T^2)/\,\mathbb{E}(T)^2 = 1 + \tfrac{256-\tau}{255\tau}(256(\|t\|_4/\|t\|_2)^4 - 1)\,.$$

Next, the Khintchine inequality $\mathbb{E}(|\langle C_{\mathsf{inf}}, t\rangle| \mid \mathfrak{J}) \ge \frac{1}{\sqrt{2}} T^{1/2}$ yields, by taking expectations,

$$\mathbb{E}(|\langle C_{\mathsf{inf}}, t\rangle|) \ge \tfrac{1}{\sqrt{2}}\, \mathbb{E}(T^{1/2})\,.$$

Applying the Cauchy–Schwarz inequality first to $T^{1/4}$ and $T^{3/4}$, and then to $T^{1/2}$ and T, we obtain $\mathbb{E}(T)^2 \leq \mathbb{E}(T^{1/2}) \cdot \mathbb{E}(T^{3/2}) \leq \mathbb{E}(T^{1/2}) \cdot \mathbb{E}(T)^{1/2} \cdot \mathbb{E}(T^2)^{1/2}$, and thus

$$\mathbb{E}(|\langle C_{\mathsf{inf}}, t\rangle|) \geq \tfrac{1}{\sqrt{2}}\, \mathbb{E}(T)^{1/2} \cdot (\mathbb{E}(T^2)/\,\mathbb{E}(T)^2)^{-1/2}\,.$$

As in the proof of Lemma 60, applying Lemma 59 now completes the argument. □

Remark 62. As already mentioned in Remark 16, the experiments indicate that the upper bound $n_{\mathsf{inf,upper}}$ is not tight, leaving room for potential improvement. To obtain such an improvement, we would need to tighten Lemma 61. Recall that the key ingredient in Lemma 61 is to show

$$\mathbb{E}(|\langle C_{\mathsf{inf}}, t\rangle|) \geq \tfrac{1}{\sqrt{2}} \sqrt{\tfrac{\tau}{256}} \|t\|_2 \cdot (1 + \tfrac{256-\tau}{255\tau}(256(\|t\|_4/\|t\|_2)^4 - 1))^{-1/2} \tag{7}$$

Hence, in order to tighten Lemma 61, we need to find a stronger inequality than (7). For instance, a very mild improvement of this inequality would be

$$\mathbb{E}(|\langle C_{\mathsf{inf}}, t\rangle|) \geq \tfrac{1}{\sqrt{2}} \sqrt{\tfrac{\tau}{256}} \|t\|_2 \tag{8}$$

Computing $n_{\mathsf{inf,upper}}$ for this inequality yields results very similar to those given in Figure 4 for (7). This highlights that (8), even if correct, would amount to only a mild improvement over (7). However, it turns out that even this mild improvement (8) is impossible. Indeed, if t is the i-th standard basis vector,

$$\tfrac{\tau}{256} = \mathbb{E}(|C_i|) = \mathbb{E}(|\langle C_{\mathsf{inf}}, t\rangle|) \geq \tfrac{1}{\sqrt{2}} \sqrt{\tfrac{\tau}{256}} \|t\|_2 = \tfrac{1}{\sqrt{2}} \sqrt{\tfrac{\tau}{256}}$$

would give rise to the contradiction $\frac{\tau}{256} \geq \frac{1}{2}$. Therefore, it appears that our upper bound cannot be significantly improved using a universal inequality such as (7). In particular, tightening the upper bound seems to require an entirely different approach.

Remark 63. The function $\theta(n_{\mathsf{inf}}, t)$ appears to depend at least on the weak integer partition $\mathfrak{p}(t)$ with parts $|t_1|, \ldots, |t_{256}|$, while the functions $\theta_{\mathsf{lower}}(n_{\mathsf{inf}}, t)$ and $\theta_{\mathsf{upper}}(n_{\mathsf{inf}}, t)$ depend only on the norms $\|t\|_2$ and $\|t\|_4$. A naive evaluation of $\Theta(n_{\mathsf{inf}}) = \#\mathcal{X} \cdot \mathbb{E}(\theta(n_{\mathsf{inf}}, X - X^*))$ therefore entails computing the distribution of the random variable $\mathfrak{p}(X - X^*)$, which is computationally demanding. To evaluate $\Theta_{\mathsf{lower}}(n_{\mathsf{inf}}) = \#\mathcal{X} \cdot \mathbb{E}(\theta_{\mathsf{lower}}(n_{\mathsf{inf}}, X - X^*))$ and $\Theta_{\mathsf{upper}}(n_{\mathsf{inf}}) = \#\mathcal{X} \cdot \mathbb{E}(\theta_{\mathsf{upper}}(n_{\mathsf{inf}}, X - X^*))$, on the other hand, it suffices to determine the joint distribution of $\|X - X^*\|_2$ and $\|X - X^*\|_4$, which is considerably more tractable.

As a consequence, it is easy to compute $n_{\mathsf{inf,lower}}$ and $n_{\mathsf{inf,upper}}$, but determining the exact value of the smallest n_{inf} for which $\mathbb{E}(\#\mathfrak{X}_{\mathsf{cand,inf}}) < 2$ is more challenging.

Heuristic extension to the low-λ regime

Remark 64. To obtain the bounds in Lemma 60 and Lemma 61, we had to restrict to the high-λ regime. This restriction was necessary as our analysis relies on the identity

$$\mathbb{P}(\mathrm{bit}_\lambda(Z_{\mathsf{inf}} - \langle C_{\mathsf{inf}}, x^*\rangle) \neq \mathrm{bit}_\lambda(Z_{\mathsf{inf}} - \langle C_{\mathsf{inf}}, x\rangle)) = \tfrac{1}{2\beta}\, \mathbb{E}(|\langle C_{\mathsf{inf}}, x - x^*\rangle|)\,,$$

derived in the proof of Lemma 59. Note that the constant 2β arises as the cardinality of the set $\mathcal{Z}_{\mathsf{inf}} \bmod^{\mp} 2^\lambda$ in the high-λ regime. By Lemma 49, this set has cardinality 2^λ in the low-λ regime. Therefore, for λ not much below $\log_2(\beta)$ in the low-λ regime, we expect Lemma 60 and Lemma 61, with 2β replaced by 2^λ, to provide good heuristic bounds, in terms of which we can extend the definitions of $n_{\mathsf{inf,lower}}$ and $n_{\mathsf{inf,upper}}$ to this regime. Simulations indicate that these heuristics are accurate when $\lambda \geq 4$ for ML-DSA-44 and when $\lambda \geq 5$ for ML-DSA-65 and ML-DSA-87. Our experimental results also align closely with these heuristics, as evidenced in Figure 1. For lower bit indices, our heuristics and experimental results start to diverge. It is an open problem, whether the heuristics or our attacks become inadequate in this regime.

A.5 Expected number of most likely keys

In the last subsection, we have focused on exact leakage, i.e. the case $p_{\mathsf{err}} = 0$. We now aim to extend our results to the noisy setting. The goal is therefore to estimate the expected number of elements in the random set

$$\mathfrak{X}_{\mathsf{ml,inf}} \coloneqq \Xi_{\mathsf{ml}}\big((C_{\mathsf{inf}}{}^{(i)}, Z_{\mathsf{inf}}{}^{(i)}, \widetilde{Y}_{\lambda,\mathsf{inf}}{}^{(i)})_{i=1}^{n_{\mathsf{inf}}}\big)\,.$$

The following theorem generalizes Lemma 61 and holds in both the exact and noisy setting. As in Section A.4, we denote by X an independent random key.

Theorem 65. *Assuming $\lambda > \log_2(\beta) + 1$, we have*

$$\mathbb{E}\left(\#\mathfrak{X}_{\mathsf{ml,inf}}\right) \le \Theta_{\mathsf{upper,noisy}}(n_{\mathsf{inf}}, p_{\mathsf{err}})\,, \tag{9}$$

where $\Theta_{\mathsf{upper,noisy}}(n_{\mathsf{inf}}, p_{\mathsf{err}}) = \#\mathcal{X}\cdot\mathbb{E}\big(\theta_{\mathsf{upper}}(n_{\mathsf{inf}}, (X-X^*)\cdot(1-2\sqrt{(1-p_{\mathsf{err}})p_{\mathsf{err}}}))\big)$ with θ_{upper} the function defined in Lemma 61.

Proof. If $p_{\mathsf{err}} = 0$, then (9) is exactly the assertion of Lemma 61. We may therefore assume that $p_{\mathsf{err}} > 0$. Let

$$W(x) = \textstyle\sum_{i=1}^{n_{\mathsf{inf}}} 1_{\widetilde{Y}_{\lambda,\mathsf{inf}}{}^{(i)} \neq \mathrm{bit}_\lambda(Z_{\mathsf{inf}}{}^{(i)} - \langle C_{\mathsf{inf}}{}^{(i)}, x\rangle)}\,.$$

Note that $\mathfrak{X}_{\mathsf{ml,inf}}$ is the set of all $x \in \mathcal{X}$ for which $W(x)$ is minimal. Then, by similar arguments as we used in Lemma 57,

$$\mathbb{E}(\#\mathfrak{X}_{\mathsf{ml,inf}}) = \textstyle\sum_{x\in\mathcal{X}} \mathbb{P}\big(\bigcap_{x'\in\mathcal{X}}\{W(x) \le W(x')\}\big) = \#\mathcal{X}\cdot\mathbb{P}\big(\bigcap_{x'\in\mathcal{X}}\{W(X) \le W(x')\}\big).$$

Moreover, by conditioning on X^*, the probability on the right-hand side is equal to

$$\begin{aligned}\textstyle\sum_{x^*\in\mathcal{X}} \mathbb{P}\big(\bigcap_{x'\in\mathcal{X}}\{W(X) \le W(x')\}, X^* = x^*\big) &\le \textstyle\sum_{x^*\in\mathcal{X}} \mathbb{P}\big(W(X) \le W(x^*),\ X^* = x^*\big)\\ &= \mathbb{P}\big(W(X) \le W(X^*)\big)\,.\end{aligned}$$

Defining $\Delta^{(i)} = 1_{\widetilde{Y}_{\lambda,\mathsf{inf}}{}^{(i)} \neq \mathrm{bit}_\lambda(Z_{\mathsf{inf}}{}^{(i)} - \langle C_{\mathsf{inf}}{}^{(i)}, X\rangle)} - 1_{\widetilde{Y}_{\lambda,\mathsf{inf}}{}^{(i)} \neq \mathrm{bit}_\lambda(Z_{\mathsf{inf}}{}^{(i)} - \langle C_{\mathsf{inf}}{}^{(i)}, X^*\rangle)}$, we have for this last probability $\mathbb{P}\big(W(X) \le W(X^*)\big) = \mathbb{P}\big(\sum_{i=1}^{n_{\mathsf{inf}}} \Delta^{(i)} \le 0\big)$ and it remains to show that

$$\mathbb{P}\big(\textstyle\sum_{i=1}^{n_{\mathsf{inf}}} \Delta^{(i)} \le 0\big) \le \mathbb{E}\big(\theta_{\mathsf{upper}}(n_{\mathsf{inf}}, (X - X^*)\cdot(1 - 2\sqrt{(1-p_{\mathsf{err}})p_{\mathsf{err}}})\big)\,.$$

We proceed similarly as in the case of exact leakage and rely on the fact that, conditioned on (X, X^*), the family $\{\Delta^{(i)}\}_{i=1}^{n_{\mathsf{inf}}}$ is independent. However, here our estimates are based on the following application of the *exponential Markov inequality* for $s > 0$:

$$\begin{aligned}\mathbb{P}\big(\textstyle\sum_{i=1}^{n_{\mathsf{inf}}} \Delta^{(i)} \le 0\big) &= \mathbb{E}\big(\mathbb{P}\big(\textstyle\sum_{i=1}^{n_{\mathsf{inf}}} \Delta^{(i)} \le 0 \mid (X, X^*)\big)\big)\\ &\le \mathbb{E}\big(\mathbb{E}\big(e^{-s\sum_{i=1}^{n_{\mathsf{inf}}}\Delta^{(i)}} \mid (X, X^*)\big)\big) = \mathbb{E}\big(\mathbb{E}\big(e^{-s\Delta} \mid (X, X^*)\big)^{n_{\mathsf{inf}}}\big),\end{aligned}$$

where we have used the conditional independence and identical distribution of the members of $\{\Delta^{(i)}\}_{i=1}^{n_{\mathsf{inf}}}$ in the last step and, for simplicity, wrote Δ for $\Delta^{(1)}$. Similarly, we will omit the superscript "(1)" for the other variables. We compute the inner expectation as

$$\mathbb{E}\left(e^{-s\Delta} \mid (X, X^*)\right) = \rho_{X,X^*} + (1-\rho_{X,X^*})(1-p_{\mathsf{err}})e^{-s} + (1-\rho_{X,X^*})p_{\mathsf{err}}e^{s}$$

where $\rho_{X,X^*} = \mathbb{P}\big(\mathrm{bit}_\lambda(Z_{\mathsf{inf}} - \langle C_{\mathsf{inf}}, X^*\rangle) = \mathrm{bit}_\lambda(Z_{\mathsf{inf}} - \langle C_{\mathsf{inf}}, X\rangle) \mid (X, X^*)\big)$. The optimal value for s is $\frac{1}{2}\log((1-p_{\mathsf{err}})/p_{\mathsf{err}})$, which lies in $(0,\infty)$ since $p_{\mathsf{err}} \in (0, 1/2)$. Thus,

$$\begin{aligned}\mathbb{P}\big(\textstyle\sum_{i=1}^{n_{\mathsf{inf}}} \Delta^{(i)} \le 0\big) &\le \mathbb{E}\big(\big(\rho_{X,X^*} + (1-\rho_{X,X^*})\cdot 2\sqrt{(1-p_{\mathsf{err}})p_{\mathsf{err}}}\,\big)^{n_{\mathsf{inf}}}\big)\\ &= \mathbb{E}\big(\big(1 - \tfrac{1}{2\beta}\,\mathbb{E}(|\langle C_{\mathsf{inf}}, X - X^*\rangle|)\cdot\big(1 - 2\sqrt{(1-p_{\mathsf{err}})p_{\mathsf{err}}}\big)\big)^{n_{\mathsf{inf}}}\big)\,,\end{aligned}$$

where we have used in the last step that $\rho_{X,X^*} = 1 - \frac{1}{2\beta}\,\mathbb{E}(|\langle C_{\mathsf{inf}}, X - X^*\rangle|)$ with the same arguments as in Section A.4. Applying (7) concludes the proof. □

Number of signatures required to recover a unique key

Remark 66. We are also interested here in the number of signatures required for our attacks. However, in this setting we are only given the upper bound (9), which relies on the estimate (7) and which, as seen in Section A.4, substantially overestimates the values observed experimentally; compare also Figure 5 for a plot of

$$n_{\mathsf{inf},\mathsf{upper},\mathsf{noisy}}(p_{\mathsf{err}}) \coloneqq \min\{n_{\mathsf{inf}} \in \mathbb{N} \mid \Theta_{\mathsf{upper},\mathsf{noisy}}(n_{\mathsf{inf}}, p_{\mathsf{err}}) < 2\}\,.$$

Therefore, in the following we instead rely on the heuristic

$$\rho_{X,X^*} = 1 - \tfrac{1}{2\beta}\,\mathbb{E}(|\langle C_{\mathsf{inf}}, X - X^*\rangle|) \approx 1 - \tfrac{1}{2\beta}\sqrt{\tfrac{\tau}{256}}\|X - X^*\|_2\,.$$

In this way, we obtain, in place of (9), the estimate

$$\mathbb{E}\,(\#\mathfrak{X}_{\mathsf{ml},\mathsf{inf}}) \lesssim \Theta_{\mathsf{noisy}}(n_{\mathsf{inf}}, p_{\mathsf{err}})\,,$$

where $\Theta_{\mathsf{noisy}}(n_{\mathsf{inf}}, p_{\mathsf{err}}) = \#\mathcal{X}\cdot\mathbb{E}\big(\big(1-\frac{1}{2\beta}\sqrt{\frac{\tau}{256}}\|X-X^*\|_2\cdot\big(1-2\sqrt{(1-p_{\mathsf{err}})p_{\mathsf{err}}}\big)\big)^{n_{\mathsf{inf}}}\big)$. Based on this, we define

$$n_{\mathsf{inf},\mathsf{noisy}}(p_{\mathsf{err}}) \coloneqq \min\{n_{\mathsf{inf}} \in \mathbb{N} \mid \Theta_{\mathsf{noisy}}(n_{\mathsf{inf}}, p_{\mathsf{err}}) < 2\}$$

as an estimate for the expected number of informative signatures required for successful key recovery in the noisy setting. This function is shown in Figure 5 for the different ML-DSA parameter sets. We have approximately $n_{\mathsf{inf},\mathsf{noisy}}(p_{\mathsf{err}}) \approx n_{\mathsf{inf},\mathsf{lower}}/\big(1-2\sqrt{(1-p_{\mathsf{err}})p_{\mathsf{err}}}\big)$ and $n_{\mathsf{inf},\mathsf{upper},\mathsf{noisy}}(p_{\mathsf{err}}) \approx n_{\mathsf{inf},\mathsf{upper}}/\big(1-2\sqrt{(1-p_{\mathsf{err}})p_{\mathsf{err}}}\big)$.

A.6 Open problems

The following questions remain as objects of investigation for future research:

1. Can the results of Section A.4 and Section A.5 be rigorously extended to the low-λ regime? See Remark 64.
2. For noisy leakage with $p_{\mathsf{err}} \gg 0$, the estimate $n_{\mathsf{inf},\mathsf{upper},\mathsf{noisy}}(p_{\mathsf{err}})$ exceeds the number of informative signatures required in the experiments by a substantially larger factor than $n_{\mathsf{inf},\mathsf{upper}}$ does in the case of exact leakage. Can the estimate be improved?
3. As outlined in Remark 62, we do not believe that the bounds derived in Section A.4 can be improved significantly using universal inequalities. However, there remains room for improvement in Θ_{upper}. For example, might partitioning $\mathrm{supp}(X - X^*)$ and deriving separate lower and upper bounds on each part yield tighter bounds?
4. Can $\min\{n_{\mathsf{inf}} \in \mathbb{N} \mid \mathbb{E}(\#\mathfrak{X}_{\mathsf{cand},\mathsf{inf}}) < 2\}$ be computed exactly rather than merely be bounded? For ML-DSA-44 and ML-DSA-87 in particular, this may be feasible.

References

[BKM26] Marcus Brinkmann, Nicolai Kraus, and Alexander May. "Halfspace Learning for Lattice Signature Key Recovery from Signs". In: *Advances in Cryptology - CRYPTO 2026 - 46th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-20, 2026, Proceedings, Part III*. Ed. by Nadia Heninger and Mike Rosulek. Vol. 16802. Lecture Notes in Computer Science. Springer, 2026, pp. 359–390. DOI: 10.1007/978-3-032-35377-1_12.

[BP18] Leon Groot Bruinderink and Peter Pessl. "Differential Fault Attacks on Deterministic Lattice Signatures". In: *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2018.3 (2018), pp. 21–43. DOI: 10.13154/TCHES.V2018.I3.21-43.

[Bro+24] Olivier Bronchain, Melissa Azouaoui, Mohamed ElGhamrawy, Joost Renes, and Tobias Schneider. "Exploiting Small-Norm Polynomial Multiplication with Physical Attacks Application to CRYSTALS-Dilithium". In: *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2024.2 (2024), pp. 359–383. DOI: 10.46586/TCHES.V2024.I2.359-383.

[CP-SAT] Laurent Perron and Frédéric Didier. *CP-SAT*. Version v9.12. Google, Feb. 17, 2025. URL: https://developers.google.com/optimization/cp/cp_solver/.

[CT06] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. 2nd. John Wiley & Sons, 2006. ISBN: 978-0-471-24195-9.

[Dam+25a] Simon Damm, Nicolai Kraus, Alexander May, Julian Nowakowski, and Jonas Thietke. "One Bit to Rule Them All - Imperfect Randomness Harms Lattice Signatures". In: *Public-Key Cryptography - PKC 2025 - 28th IACR International Conference on Practice and Theory of Public-Key Cryptography, Røros, Norway, May 12-15, 2025, Proceedings, Part I*. Ed. by Tibor Jager and Jiaxin Pan. Vol. 15674. Lecture Notes in Computer Science. Springer, 2025, pp. 284–316. DOI: 10.1007/978-3-031-91820-9_10.

[Dam+25b] Simon Damm, Asja Fischer, Alexander May, Soundes Marzougui, Leander Schwarz, Henning Seidler, Jean-Pierre Seifert, Jonas Thietke, and Vincent Quentin Ulitzsch. "Solving Concealed ILWE and Its Application for Breaking Masked Dilithium". In: *Advances in Cryptology - ASIACRYPT 2025 - 31st International Conference on the Theory and Application of Cryptology and Information Security, Melbourne, VIC, Australia, December 8-12, 2025, Proceedings, Part II*. Ed. by Goichiro Hanaoka and Bo-Yin Yang. Lecture Notes in Computer Science. Springer, 2025, pp. 608–640. DOI: 10.1007/978-981-95-5096-8_19.

[Dam+26] Simon Damm, Nicolai Kraus, Alexander May, Julian Nowakowski, and Jonas Thietke. "One (Noisy) Bit to Rule Them All: Key Recovery from Randomness Leakage in ML-DSA". In: *J. Cryptol.* 39.4 (2026), p. 28. DOI: 10.1007/S00145-026-09584-7.

[Dilithium] Shi Bai, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. *CRYSTALS-Dilithium: Algorithm Specification and Supporting Documentation (Version 3.1)*. Round 3 Submission to the NIST Post-Quantum Cryptography Standardization Project. See https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf. 2021.

[Duc+18] Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. "CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme". In: *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2018.1 (2018), pp. 238–268. DOI: 10.13154/TCHES.V2018.I1.238-268.

[FIPS204] National Institute of Standards and Technology. *Module-Lattice-Based Digital Signature Standard*. Federal Information Processing Standards Publication (FIPS) NIST FIPS 204. Washington, D.C.: U.S. Department of Commerce, 2024. DOI: 10.6028/NIST.FIPS.204.

[Gurobi] Gurobi Optimization, LLC. *Gurobi Optimizer Reference Manual*. 2025. URL: https://www.gurobi.com.

[Heaviside] Wikipedia contributors. *Heaviside step function — Wikipedia, The Free Encyclopedia*. 2026. URL: https://en.wikipedia.org/w/index.php?title=Heaviside_step_function&oldid=1340345270.

[Her+25] Julius Hermelink, Silvan Streit, Erik Mårtensson, and Richard Petri. "A Generic Framework for Side-Channel Attacks Against LWE-Based Cryptosystems". In: *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques Madrid, Spain, May 4-8, 2025, Proceedings, Part VIII*. Ed. by Serge Fehr and Pierre-Alain Fouque. Vol. 15608. Lecture Notes in Computer Science. Springer, 2025, pp. 3–32. DOI: 10.1007/978-3-031-91101-9_1.

[HiGHS] Qi Huangfu and J. A. J. Hall. "Parallelizing the dual revised simplex method". In: *Math. Program. Comput.* 10.1 (2018), pp. 119–142. DOI: 10.1007/S12532-017-0130-5.

[HNP25] Julius Hermelink, Kai-Chun Ning, and Richard Petri. "Finding and Protecting the Weakest Link – On Side-Channel Attacks on $\boldsymbol{y}$ in Masked ML-DSA". In: *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part V*. Ed. by Yael Tauman Kalai and Seny F. Kamara. Lecture Notes in Computer Science. Springer, 2025, pp. 3–37. DOI: 10.1007/978-3-032-01901-1_1.

[HRG14] Annelie Heuser, Olivier Rioul, and Sylvain Guilley. "Good Is Not Good Enough - Deriving Optimal Distinguishers from Communication Theory". In: *Cryptographic Hardware and Embedded Systems - CHES 2014 - 16th International Workshop, Busan, South Korea, September 23-26, 2014. Proceedings*. Ed. by Lejla Batina and Matthew Robshaw. Vol. 8731. Lecture Notes in Computer Science. Springer, 2014, pp. 55–74. DOI: 10.1007/978-3-662-44709-3_4.

[Julia] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B Shah. "Julia: A fresh approach to numerical computing". In: *SIAM Review* 59.1 (2017), pp. 65–98. DOI: 10.1137/141000671.

[JuMP] Miles Lubin, Oscar Dowson, Joaquim Dias Garcia, Joey Huchette, Benoît Legat, and Juan Pablo Vielma. "JuMP 1.0: Recent improvements to a modeling language for mathematical optimization". In: *Mathematical Programming Computation* (2023). DOI: 10.1007/s12532-023-00239-3.

[Liu+21] Yuejun Liu, Yongbin Zhou, Shuo Sun, Tianyu Wang, Rui Zhang, and Jingdian Ming. "On the Security of Lattice-Based Fiat-Shamir Signatures in the Presence of Randomness Leakage". In: *IEEE Trans. Inf. Forensics Secur.* 16 (2021), pp. 1868–1879. DOI: 10.1109/TIFS.2020.3045904.

[Lyu09] Vadim Lyubashevsky. "Fiat-Shamir with Aborts: Applications to Lattice and Factoring-Based Signatures". In: *Advances in Cryptology - ASIACRYPT 2009, 15th International Conference on the Theory and Application of Cryptology and Information Security, Tokyo, Japan, December 6-10, 2009. Proceedings*. Ed. by Mitsuru Matsui. Vol. 5912. Lecture Notes in Computer Science. Springer, 2009, pp. 598–616. DOI: 10.1007/978-3-642-10366-7_35.

[Lyu24] Vadim Lyubashevsky. "Basic Lattice Cryptography: The concepts behind Kyber (ML-KEM) and Dilithium (ML-DSA)". In: *IACR Cryptol. ePrint Arch.* (2024), p. 1287. URL: https://eprint.iacr.org/archive/2024/1287/20250618:125932.

[Oli+25] Paco Azevedo Oliveira, Andersson Calle Viera, Benoît Cogliati, and Louis Goubin. "Uncompressing Dilithium's Public Key". In: *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part I*. Ed. by Yael Tauman Kalai and Seny F. Kamara. Vol. 16000. Lecture Notes in Computer Science. Springer, 2025, pp. 417–443. DOI: 10.1007/978-3-032-01855-7_14.

[Optim.jl] Patrick Kofod Mogensen and Asbjørn Nilsen Riseth. "Optim: A mathematical optimization package for Julia". In: *Journal of Open Source Software* 3.24 (2018), p. 615. DOI: 10.21105/joss.00615.

[Qia+23] Zehua Qiao, Yuejun Liu, Yongbin Zhou, Jingdian Ming, Chengbin Jin, and Huizhong Li. "Practical Public Template Attack Attacks on CRYSTALS-Dilithium With Randomness Leakages". In: *IEEE Trans. Inf. Forensics Secur.* 18 (2023), pp. 1–14. DOI: 10.1109/TIFS.2022.3215913.

[Qia+24] Zehua Qiao, Yuejun Liu, Yongbin Zhou, Yuhan Zhao, and Shuyi Chen. "Single Trace is All It Takes: Efficient Side-channel Attack on Dilithium". In: *IACR Cryptol. ePrint Arch.* (2024), p. 512. URL: https://eprint.iacr.org/archive/2024/512/20240414:015419.

[Rav+18] Prasanna Ravi, Mahabir Prasad Jhanwar, James Howe, Anupam Chattopadhyay, and Shivam Bhasin. "Side-channel Assisted Existential Forgery Attack on Dilithium - A NIST PQC candidate". In: *IACR Cryptol. ePrint Arch.* (2018), p. 821. URL: https://eprint.iacr.org/archive/2018/821/20180916:023812.

[Sch+26] Carsten Schubert, Niklas Julius Müller, Jean-Pierre Seifert, and Marian Margraf. "Descent into Broken Trust: Uncovering ML-DSA Subkeys with Scarce Leakage and Local Optimization". In: *IACR Cryptol. ePrint Arch.* (2026), p. 472. URL: https://eprint.iacr.org/archive/2026/472/20260306:220315.

[SCIP] Suresh Bolusani, Mathieu Besançon, Ksenia Bestuzheva, Antonia Chmiela, João Dionísio, Tim Donkiewicz, Jasper van Doornmalen, Leon Eifler, Mohammed Ghannam, Ambros Gleixner, Christoph Graczyk, Katrin Halbig, Ivo Hedtke, Alexander Hoen, Christopher Hojny, Rolf van der Hulst, Dominik Kamp, Thorsten Koch, Kevin Kofler, Jurgen Lentz, Julian Manns, Gioni Mexi, Erik Mühmer, Marc E. Pfetsch, Franziska Schlösser, Felipe Serrano, Yuji Shinano, Mark Turner, Stefan Vigerske, Dieter Weninger, and Lixing Xu. *The SCIP Optimization Suite 9.0*. Technical Report. Optimization Online, Feb. 2024. URL: https://optimization-online.org/2024/02/the-scip-optimization-suite-9-0/.

[Uli+22] Vincent Quentin Ulitzsch, Soundes Marzougui, Mehdi Tibouchi, and Jean-Pierre Seifert. "Profiling Side-Channel Attacks on Dilithium - A Small Bit-Fiddling Leak Breaks It All". In: *Selected Areas in Cryptography - 29th International Conference, SAC 2022, Windsor, ON, Canada, August 24-26, 2022, Revised Selected Papers*. Ed. by Benjamin Smith and Huapeng Wu. Vol. 13742. Lecture Notes in Computer Science. Springer, 2022, pp. 3–32. DOI: 10.1007/978-3-031-58411-4_1.

[Wan+21] Tianyu Wang, Yuejun Liu, Jun Xu, Lei Hu, Yang Tao, and Yongbin Zhou. "Integer LWE with Non-subgaussian Error and Related Attacks". In: *Information Security - 24th International Conference, ISC 2021, Virtual Event, November 10-12, 2021, Proceedings*. Ed. by Joseph K. Liu, Sokratis K. Katsikas, Weizhi Meng, Willy Susilo, and Rolly Intan. Vol. 13118. Lecture Notes in Computer Science. Springer, 2021, pp. 3–25. DOI: 10.1007/978-3-030-91356-4_1.